

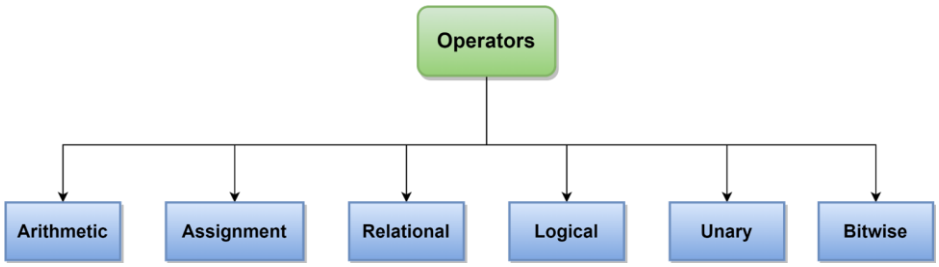
Spring-2024

CSE 110

Programming Language-I

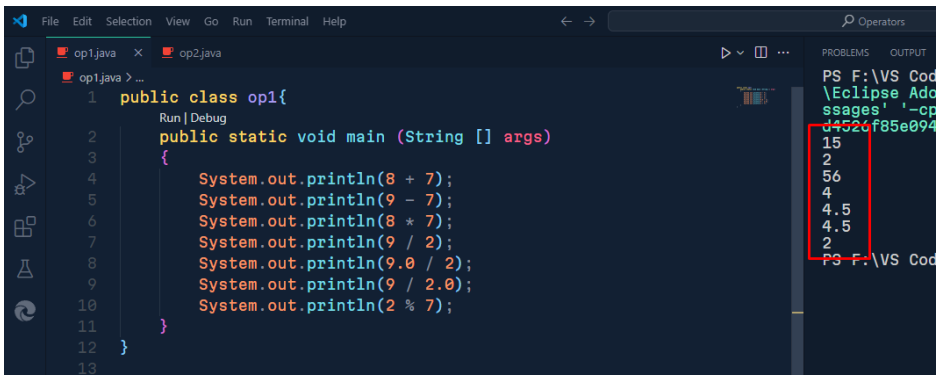
Operators

- Operators are symbols used to perform specific operations on one or more operands to produce a result.
- Types of operators in Java:



Arithmetic Operators:

Category	Name	Symbol
Additive	Addition	+
	Subtraction	-
Multiplicative	Multiplication	*
	Division	/
	Modulus	%



Modulus (%): Remainder (ভাগশেষ)

2%3 → 3) 2 (0
0
2

ছোট সংখ্যাকে বড় সংখ্যা দ্বারা ভাগ করলে ভাগশেষ হবে ছোট সংখ্যা।

Assignment Operators:

Name	Symbol	Identicality
Assignment	=	a = 5;
Addition Equals	+=	a += 3; a = a+3;
Subtraction Equals	-=	a -= 3; a = a-3;
Multiplication Equals	*=	a *= 2; a = a*2;
Division Equals	/=	a /= 2; a = a/2;
Modulus Equals	%=	a %= 2; a = a%2;



Relational Operators:

Category	Name	Symbol
Comparison	Greater Than	>
	Less Than	<
	Greater Than or Equals	>=
	Less Than or Equals	<=
Equality	Equals	==
	Not Equals	!=

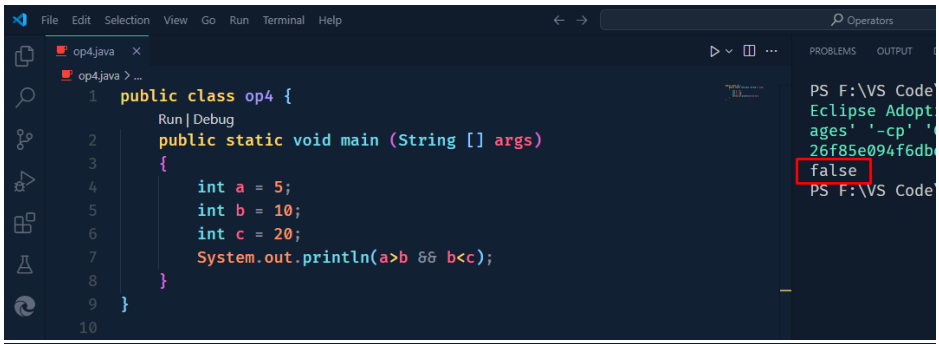


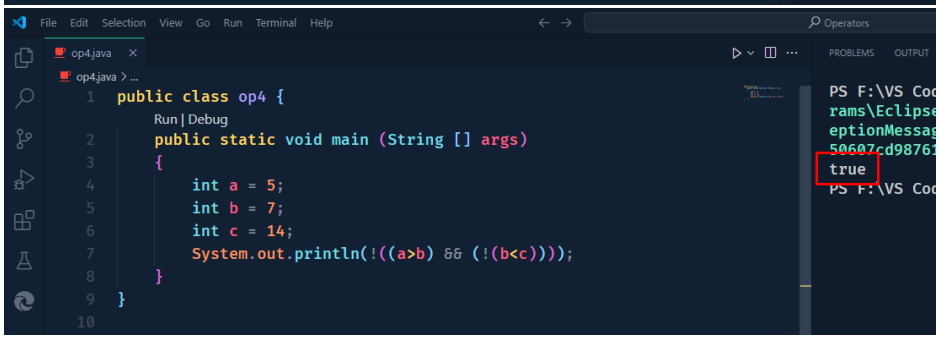
এক্ষেত্রে Boolean Value দেয়

Logical Operators:

Name	Symbol
Logical AND	&&
Logical OR	||
Logical NOT	!

AND (&&): Denotes true if both the conditions before and after are true, otherwise false.
Example:





A	B	A && B

true	true	true
true	false	false
false	true	false
false	false	false

◆ অর্থাৎ, ২ টি শর্তই true হতে হবে নয়তো output আসবে false.

OR (||): Denotes true if at least one conditions before and after are true, otherwise false.
Example:



A	B	A || B
true	true	true
true	false	true
false	true	true
false	false	false

◆ অর্থাৎ, যেকোনো ১ টি শর্ত true হলেই output আসবে true.

Unary Operators:

Type	Name	Symbol
Prefix	Negation	-
	Not	!
	Increment	++
	Decrement	--
Postfix	Increment	++
	Decrement	--

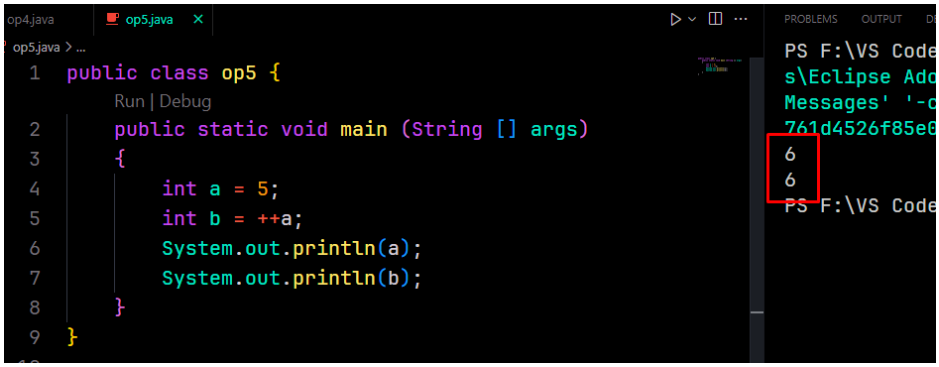
Negation (-): Negates an operand.



Not (!): Reverses the logical value of an operand.

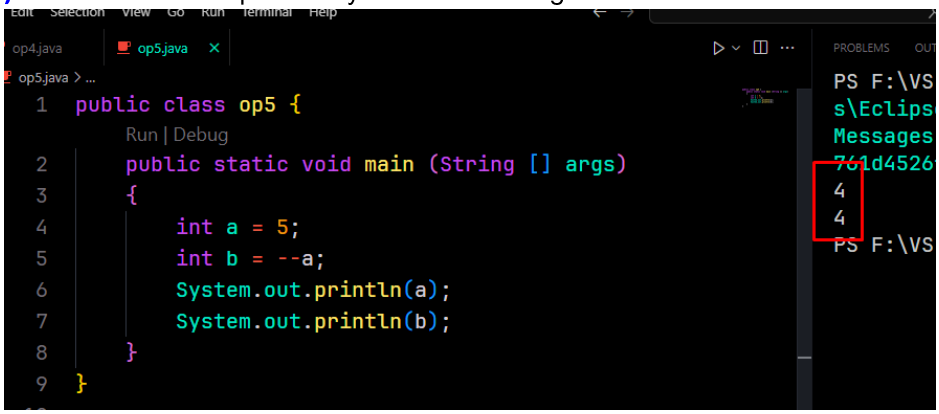


Pre-Increment (++): Increments the operand by 1 and then assigned.



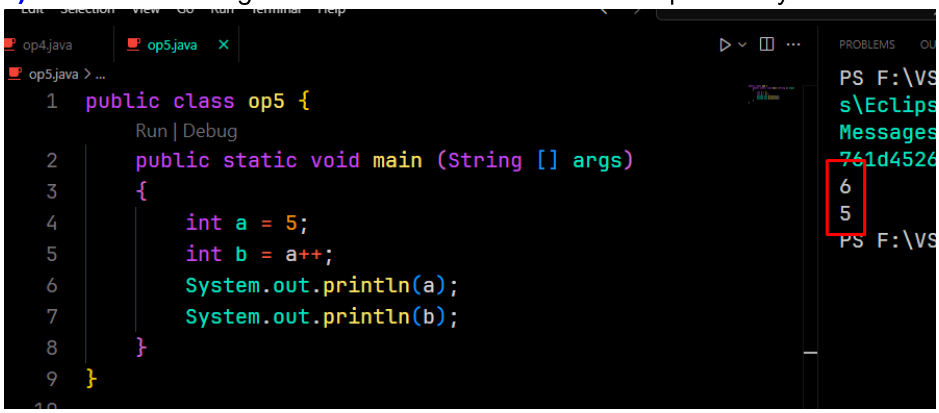
এখানে, a এর value আগে ১ বাড়বে, তারপর সেটা b তে বসে যাবে।

Pre-Decrement (--): Decrements the operand by 1 and then assigned.



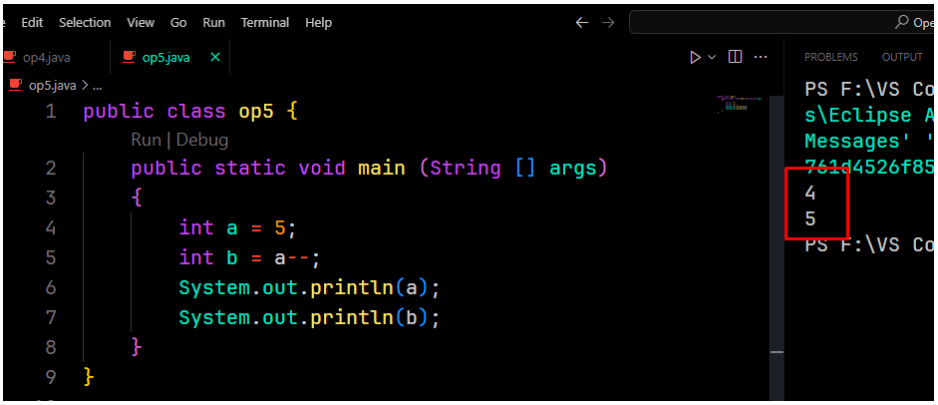
এখানে, a এর value আগে ১ কমবে, তারপর সেটা b তে বসে যাবে।

Post-Increment (++): The value is assigned first and then increments the operand by 1.



এখানে, a এর value আগে b তে বসে যাবে। তারপর a এর value ১ বাড়বে।

Post-Decrement (--): The value is assigned first and then decrements the operand by 1.



এখানে, a এর value আগে b তে বসে যাবে। তারপর a এর value ১ কমবে।

4.8 Operator Precedence

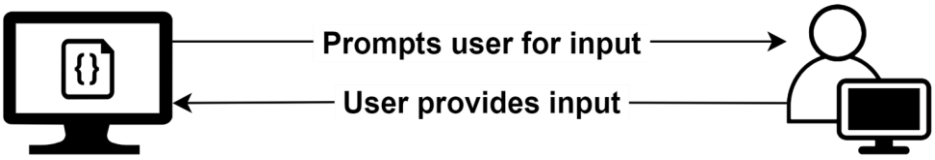
The precedence is higher at the top and decreases as we move towards the bottom. If multiple operators with the same precedence are placed adjacently, operators are executed from left to right. If the first brackets "( )" are present, we must go to the innermost first bracket and gradually move towards the outer ones.

Operator Type	Operator Category/Name	Symbol
Unary	Postfix	++, --
	Prefix	++, --, !, -
Arithmetic	Multiplicative	*, / , %
	Additive	+, -
Relational	Comparison	<, >, <=, >=
	Equality	==, !=
Bitwise	Bitwise AND	&
	Bitwise Exclusive OR	^
	Bitwise Inclusive OR	|
Logical	Logical AND	&&
	Logical OR	||
Assignment	All assignment operators	=, +=, -=, *=, /=, %=

JAVA User Input

User input in Java

- Sometimes we need to take input from the user.
- Creates a connection between the program and it's user.
- Most common way to take input is using the java scanner class.

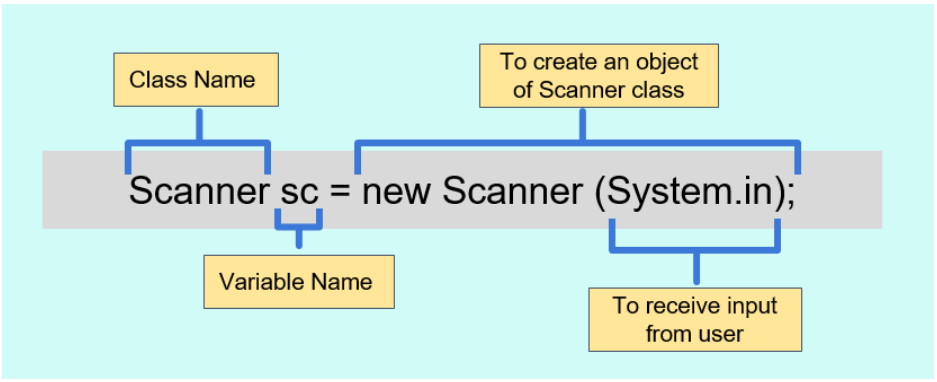


What is a Java Scanner Class?

- A predefined class that reads data from the keyboard or a text file.
- Found in the **java.util** package.
- Can read all types of data (numeric, strings and other data types).
- Easiest way to take input from the user.

User Input using Scanner class

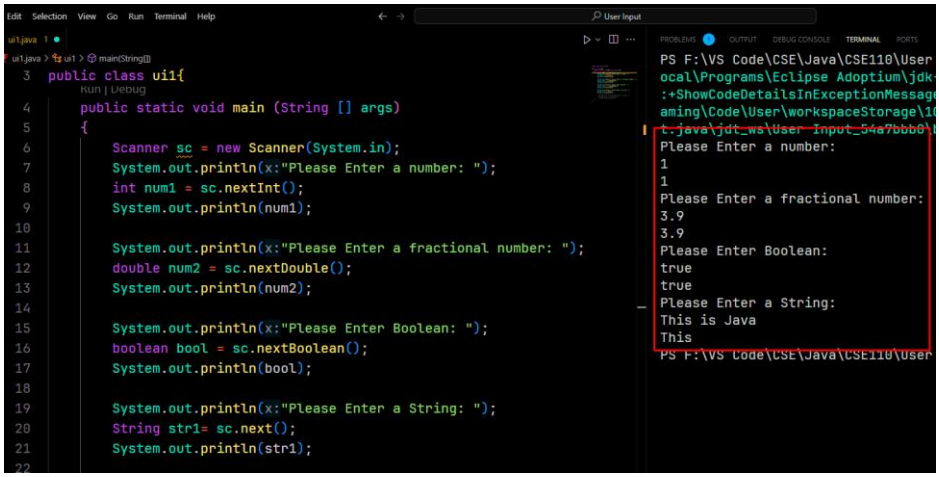
- Import the **java.util**.Scanner package
- Create an object of the Scanner class
- Syntax:



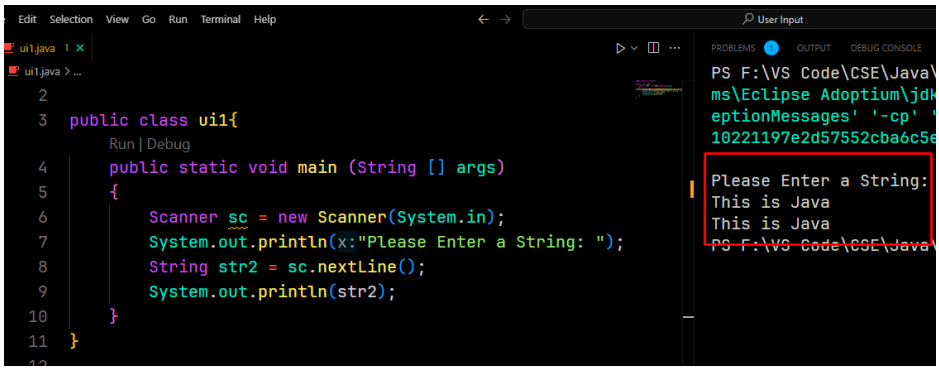
Inputting Different Datatypes

Method	Usage

nextInt( )	Reads an integer value from the user
nextLong( )	Reads a long value from the user
nextFloat( )	Reads a float value from the user
nextDouble( )	Reads a double value from the user
nextBoolean( )	Reads a boolean value from the user
nextLine( )	Reads a string value from the user



String Input নেয়ার সময় sc.next( ) এর ক্ষেত্রে প্রথম space পর্যন্ত নিবে।



sc.nextline( ) এর ক্ষেত্রে সম্পূর্ণ লাইন নিবে।

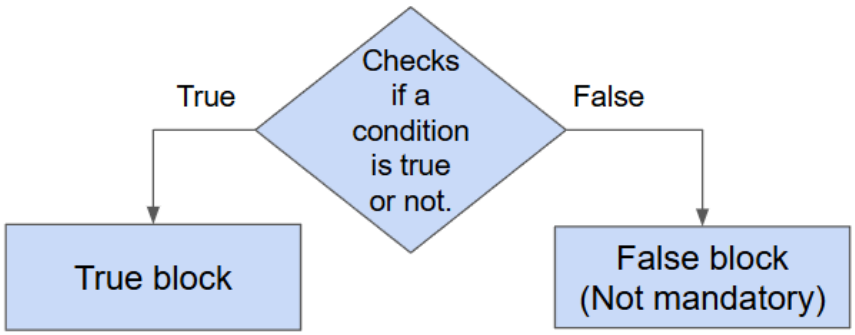
Decision Making

Branching or Selection Statements

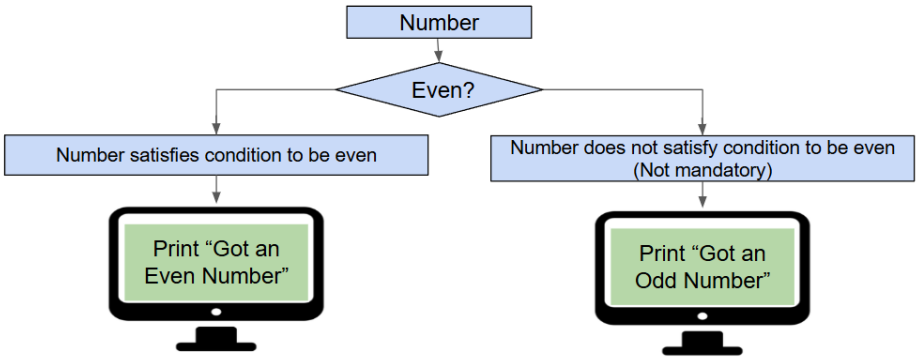
Conditional / Selection Statement

- Execution of a block of code on the basis of some **Conditions**.
- Non sequential program statements
- Selection statements may be **unary or non unary.**

Ex: if, else if, else, switch-case



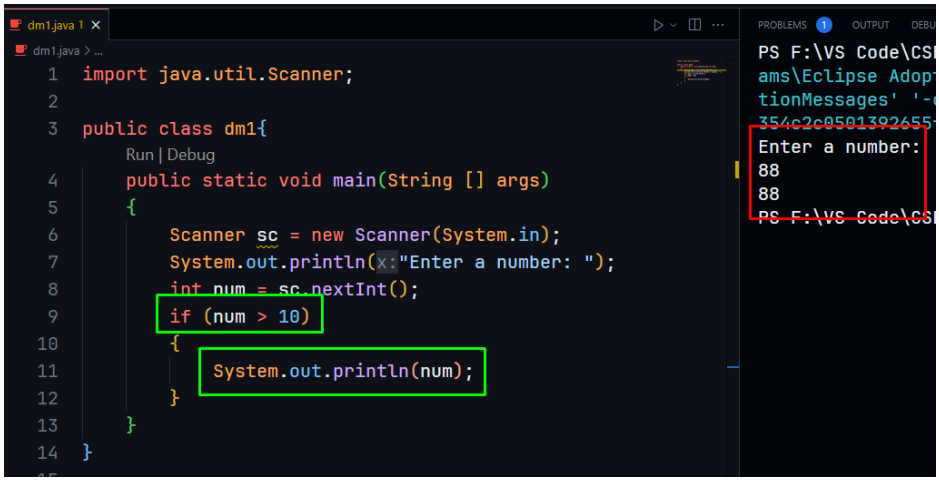
Ex: Printing “Got an Even Number” only if a number satisfies condition to be even or Print “Got an Odd Number” if a number did not satisfy condition to be even.



অর্থাৎ, ১টা number নিবো এবং যাচাই করবো সেটা Even(জোড়) কিনা। যদি Even (জোড়) হয় তাহলে বলবো Even number পেয়েছি নয়তো বলবো Odd number (বিজোড়) পেয়েছি।

if Selection Statement

- Unary selection statement
- Will execute block of code only if condition is **True**.



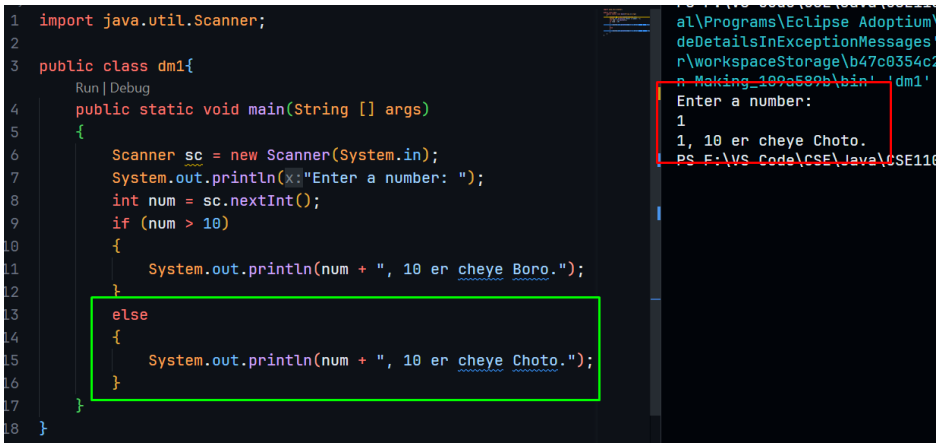
Checking Conditions using Operators

- Used to evaluate expressions and return a boolean value **(true or false).**
- Conditional operators in Java: Equality Operator **(==)**, Inequality Operator **(!=)**, Comparison Operators **(<, >, <=, >=)**, Logical Operators: NOT **(!)**, AND **(&&)**, OR **(||)**.
- Essential for creating effective branching logic.

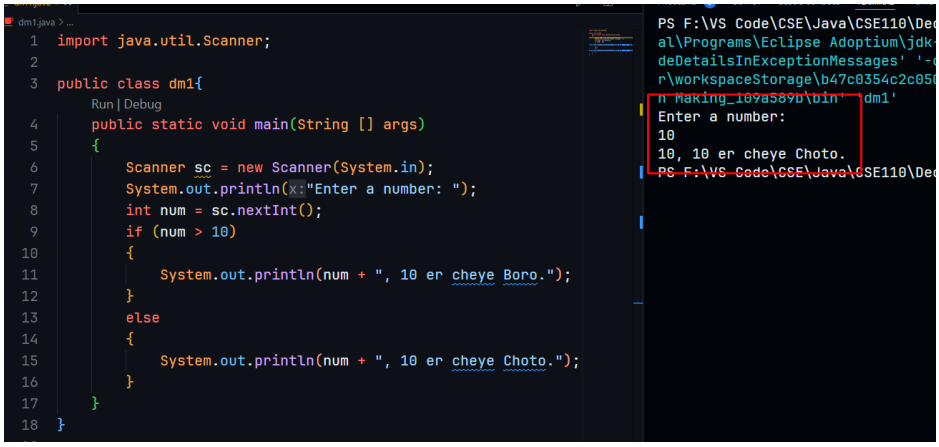
if - else Double Selection Statements

- else block executed when if condition yields **False**.
- else does not have any condition.





এখানে যদি 10 input দেই তাহলে?



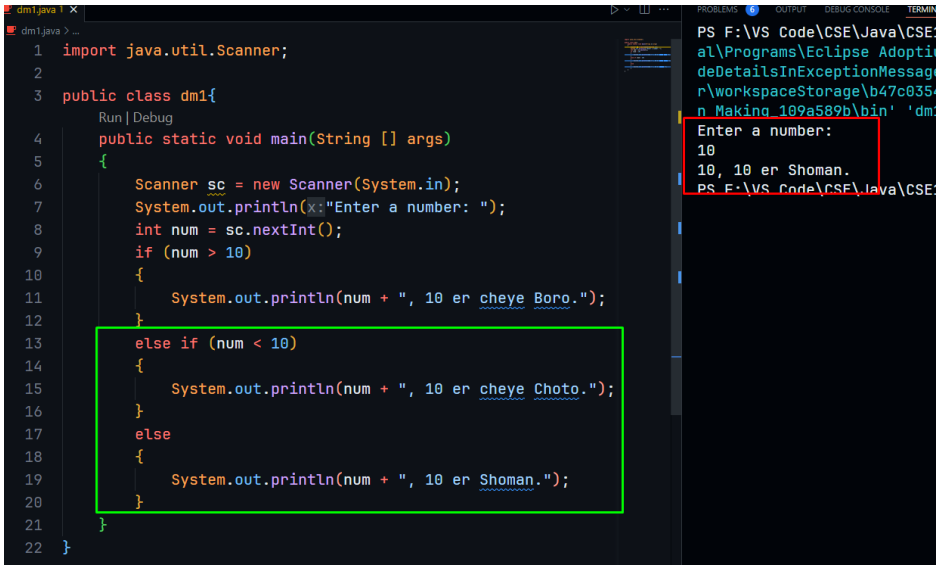
```java
import java.util.Scanner;

public class dm1{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Enter a number: ");
        int num = sc.nextInt();
        if (num > 10)
        {
            System.out.println(num + ", 10 er cheye Boro.");
        }
        else
        {
            System.out.println(num + ", 10 er cheye Choto.");
        }
    }
}
```

```
Enter a number:
10
10, 10 er cheye Choto.
```

if - else if - else Multiple Selection Statements

- Follows top down approach.
- There can be multiple else if conditions.
- else if and else cannot exist without if condition.
- Only the first True conditional block executed. Following conditions are not checked.
    - Use of elif/ else: Evaluating other conditions after if statement.



```java
import java.util.Scanner;

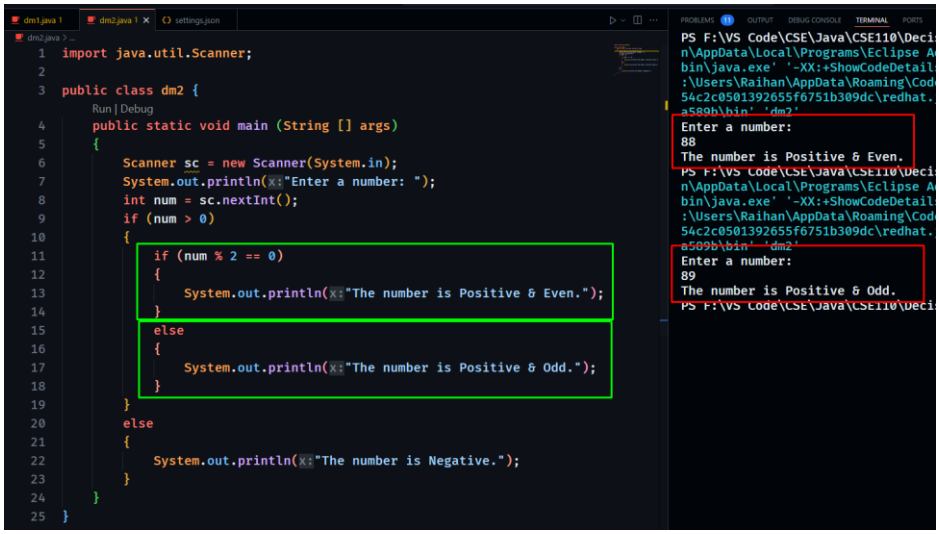
public class dm1{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Enter a number: ");
        int num = sc.nextInt();
        if (num > 10)
        {
            System.out.println(num + ", 10 er cheye Boro.");
        }
        else if (num < 10)
        {
            System.out.println(num + ", 10 er cheye Choto.");
        }
        else
        {
            System.out.println(num + ", 10 er Shoman.");
        }
    }
}
```

```
Enter a number:
10
10, 10 er Shoman.
```

Nested Decision Making

- A conditional statement contained inside another conditional statement.
- Statements inside are only evaluated if outer branching statements are true.
    - Use: Building complex layered logic.

Use case: Checking if an even number is a prime number will require nested if statements. Outer if statement will check if the number is even or not and the inner if will check if the number is prime or not.



```java
import java.util.Scanner;

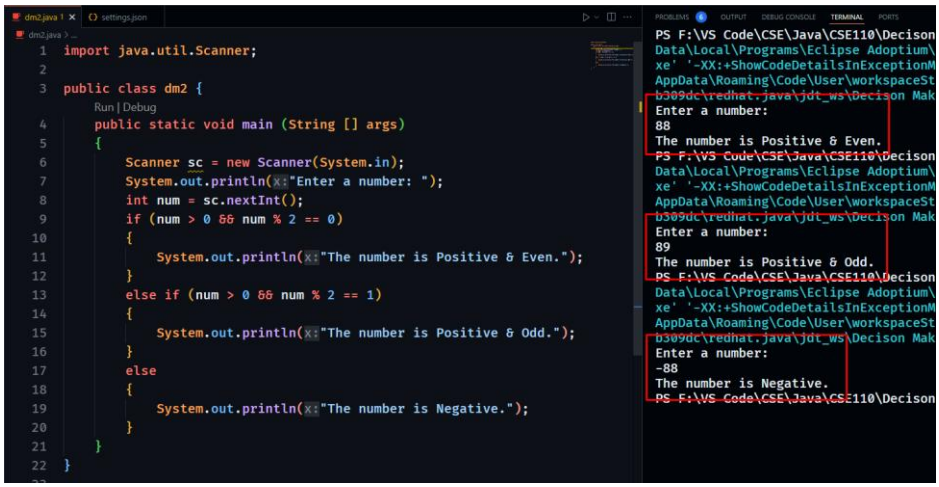
public class dm2 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number: ");
        int num = sc.nextInt();
        if (num > 0)
        {
            if (num % 2 == 0)
            {
                System.out.println("The number is Positive & Even.");
            }
            else
            {
                System.out.println("The number is Positive & Odd.");
            }
        }
        else
        {
            System.out.println("The number is Negative.");
        }
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE110\Deci
n\AppData\Local\Programs\Eclipse A
bin\java.exe' '-XX:+ShowCodeDetail
:\Users\Raihan\AppData\Roaming\Cod
54c2c0501392655f6751b309dc\redhat.
a589b\bin' 'dm2'
Enter a number:
88
The number is Positive & Even.
PS F:\VS Code\CSE\Java\CSE110\Deci
n\AppData\Local\Programs\Eclipse A
bin\java.exe' '-XX:+ShowCodeDetail
:\Users\Raihan\AppData\Roaming\Cod
54c2c0501392655f6751b309dc\redhat.
a589b\bin' 'dm2'
Enter a number:
89
The number is Positive & Odd.
PS F:\VS Code\CSE\Java\CSE110\Deci
```

এখানে মূলত Condition এর ভিতরে Condition.

Nested না করেই Code টা করা যায় কিনা?

- অবশ্যই যায়।



```java
import java.util.Scanner;

public class dm2 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number: ");
        int num = sc.nextInt();
        if (num > 0 && num % 2 == 0)
        {
            System.out.println("The number is Positive & Even.");
        }
        else if (num > 0 && num % 2 == 1)
        {
            System.out.println("The number is Positive & Odd.");
        }
        else
        {
            System.out.println("The number is Negative.");
        }
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE110\Decison
Data\Local\Programs\Eclipse Adoptium\
xe' '-XX:+ShowCodeDetailsInExceptionM
AppData\Roaming\Code\User\workspaceSt
b309dc\redhat.java\jdt_ws\Decison Mak
Enter a number:
88
The number is Positive & Even.
PS F:\VS Code\CSE\Java\CSE110\Decison
Data\Local\Programs\Eclipse Adoptium\
xe' '-XX:+ShowCodeDetailsInExceptionM
AppData\Roaming\Code\User\workspaceSt
b309dc\redhat.java\jdt_ws\Decison Mak
Enter a number:
89
The number is Positive & Odd.
PS F:\VS Code\CSE\Java\CSE110\Decison
Data\Local\Programs\Eclipse Adoptium\
xe' '-XX:+ShowCodeDetailsInExceptionM
AppData\Roaming\Code\User\workspaceSt
b309dc\redhat.java\jdt_ws\Decison Mak
Enter a number:
-88
The number is Negative.
PS F:\VS Code\CSE\Java\CSE110\Decison
```

Loops (while)

Intro to loop

Loop is one of fundamental concepts of programming.

It is a **control structure** that allows one to repeatedly execute a block of codes.

Components of a loop:

1) **Initialization:** one or more variable(s) to keep track of the loop.
2) **Condition:** boolean expression based on which the loop runs.
3) **Body:** block of codes which is executed each time the loop runs.
4) **Steps:** updating the initialized variable(s) to make sure the loop runs certain amount of times only.

1, 2, 4 পরস্পরের উপর নির্ভরশীল।

Loops in Java

There are three types of loops in Java.

1. while loop.
2. for loop.
3. do while loop.

◆ **for and while** loop only enters the loop's body if the condition is true.
◆ On the other hand, **do while** loop executes the body's code first then checks the condition.

while loop

Structure of **while loop**:

```
1.  INITIALIZATION
2.  while ( CONDITION ){
3.      BODY OF THE LOOP
4.      STEPS
5.  }
```

Key points of the structure:

- ➢ We **initialize** the variable outside the loop's body.
- ➢ After the **while** keyword we write the **condition** within parenthesis.
- ➢ Then at the end of the loop's body we **increase/decrease** the initialized variable.

```java
public class WhileLoopScenario1{
    public static void main(String[] args) {
        int i = 0;
        while ( i < 500 ) {
            System.out.println("CSE IS FUN");
            i+=1;
        }
    }
}
```

Initialization

Condition

Body

Steps

```
Welcome to DrJava.  Working directory is /home/Java110
> run WhileLoopScenario1
CSE IS FUN
CSE IS FUN
.....
```

```java
public class wl1{
    Run | Debug
    public static void main (String [] args)
    {
        int i = 0; //Initialization
        while (i < 10) //Condition
        {
            System.out.println(x:"CSE 110"); //Body
            i += 1;  //Steps
        }
    }
}
```

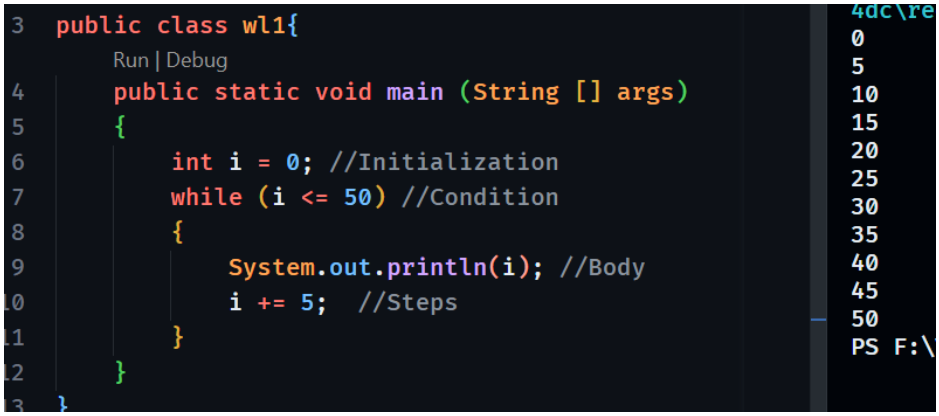
```
f46059071e6cba16
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
PS F:\VS Code\CS
```

```java
public class wl1{
    Run | Debug
    public static void main (String [] args)
    {
        int i = 5; //Initialization
        while (i >= 1) //Condition
        {
            System.out.println(i); //Body
            i -= 1;  //Steps
        }
    }
}
```

```
4dc\redhat.ja
5
4
3
2
1
PS F:\VS Code
```

```java
public class wl1{
    Run | Debug
    public static void main (String [] args)
    {
        int i = 1; //Initialization
        while (i <= 50) //Condition
        {
            System.out.println(i); //Body
            i += 3;  //Steps
        }
    }
}
```

```
4dc\redhat.
1
4
7
10
13
16
19
22
25
28
31
34
37
40
43
46
49
```


public class wl1{
Run | Debug
public static void main (String [] args)
{
int i = 0; //Initialization
while (i <= 50) //Condition
{
System.out.println(i); //Body
i += 5; //Steps
}
}
}
4dc\re
0
5
10
15
20
25
30
35
40
45
50
PS F:\

Loops (for loop)

for loop

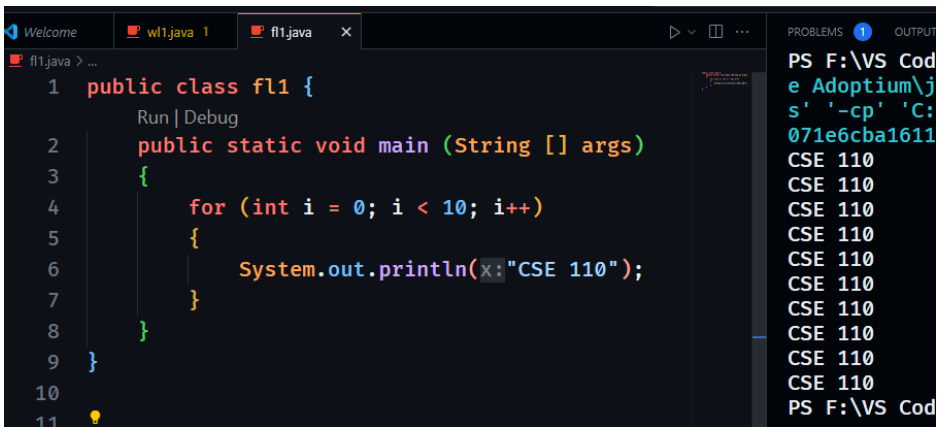
Structure of **for loop**:

```
1.  for ( INITIALIZATION ; CONDITION ; STEPS ) {
2.          BODY OF THE LOOP
3.  }
```

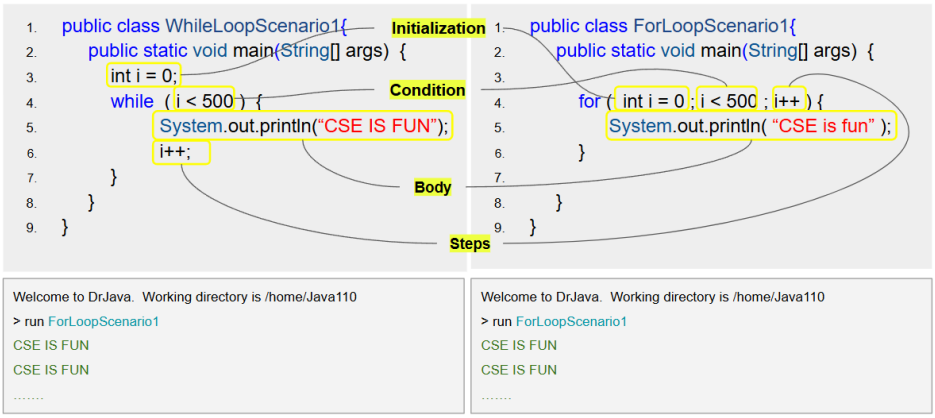
Key points of the structure:

- We **initialize** variable, write the **condition** and **increase/decrease** the initialized variable within a parenthesis right after the **for** keyword.
- The aforementioned 3 components are separated using **;** (**semicolon**) .
- We can initialize or increase/decrease multiple variables at once by separating them using **,** (**Comma**).

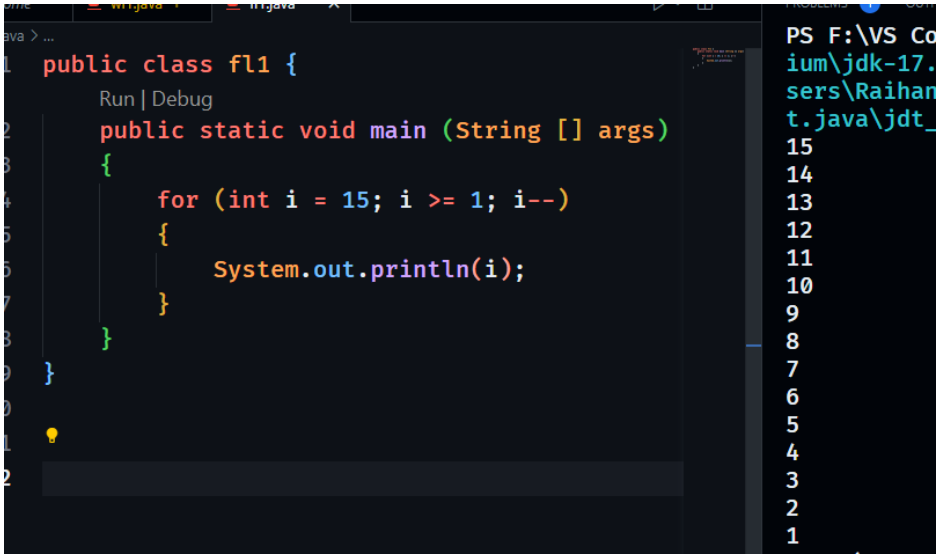
Scenario: Write a java program using for loop to print “CSE 110” 10 times.



for loop vs while loop



◆ 15 থেকে 1 পর্যন্ত উল্টা Print করতে চাইলে ->



◆ ১ থেকে ৫০ পর্যন্ত Even number (জোড় সংখ্যা) Print করতে চাইলে-



◆ User ৫ টা number দিবে, সেখান থেকে maximum value অর্থাৎ সবচেয়ে বড় সংখ্যা বের করতে হবে।



```java
import java.util.Scanner;

public class fl2 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int maxVal = 0;

        for (int i = 1; i <= 5; i++)
        {
            System.out.println(x:"Enter a number: ");
            int num = sc.nextInt();
            if (i == 1)
            {
                maxVal = num;
            }
            else
            {
                if (num > maxVal)
                {
                    maxVal = num;
                }
            }
        }
        System.out.println(maxVal + " is the maximum.");
    }
}
```

```
PS F:\VS Code\CSE\Java\CS
Eclipse Adoptium\jdk-17.0
ptionMessages' '-cp' 'C:\
\18a4976ec9df4605907 1e6cb
Enter a number:
9
Enter a number:
5
Enter a number:
17
Enter a number:
19
Enter a number:
88
88 is the maximum.
PS F:\VS Code\CSE\Java\CS
```

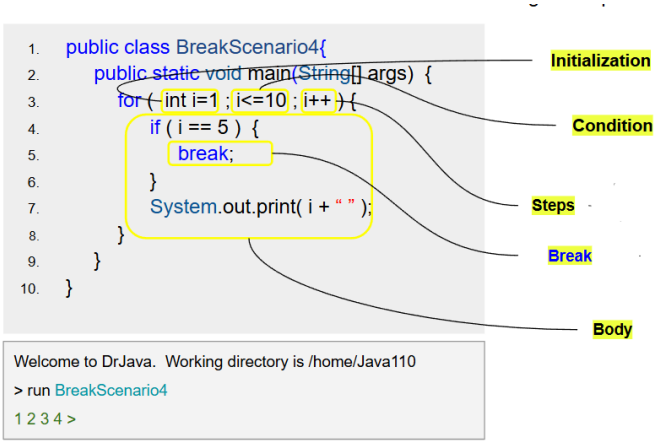
Loops (break/continue)

Two important concepts one should learn with loop are **break** & **continue**. These two keywords grant us more control when running the loop.

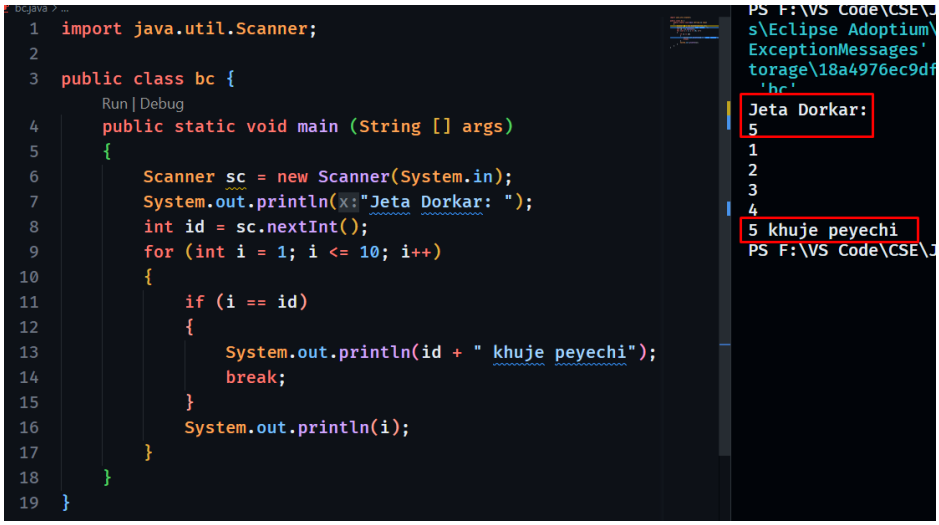
- **break:** the break keyword allows us to **stop the loop** early.
- **continue**: the continue keyword allows us to skip the rest of the loop body and start the next iteration of the loop.

Note: both break and continue are written inside the loop's body and are generally bounded by a condition.

break keyword – code



◆ break এর মূল কাজ হলো একটা Loop কে ভেঙ্গে দেওয়া।



এখানে 5 দরকার ছিল, তাই 5 পেয়ে যাবার পর break হয়ে Loop এর বাইরে চলে এসেছে।

continue keyword – code

```
1.  public class ContinueScenario4{
2.      public static void main(String[] args) {
3.          for ( int i=1 ; i<=10 ; i++ ) {
4.              if ( i == 5 ) {
5.                  continue;
6.              }
7.              System.out.print( i + " " );
8.          }
9.      }
10. }
```

Initialization

Condition

Steps

Continue

Body

```
Welcome to DrJava.  Working directory is /home/Java110
> run ContinueScenario4
1 2 3 4 6 7 8 9 10 >
```

```
bc2.java > ...
1   import java.util.Scanner;
2
3   public class bc2 {
        Run | Debug
4       public static void main (String [] args)
5       {
6           Scanner sc = new Scanner(System.in);
7           for (int i = 1; i <= 10; i++)
8           {
9               if (i == 6 || i == 8)
10              {
11                  continue;
12              }
13              System.out.println(i + " --> chocolate given.");
14          }
15      }
16  }
```

```
PS F:\VS Code\CSE\Java\C
ams\Eclipse Adoptium\jdk
lsInExceptionMessages' '
spaceStorage\18a4976ec9d
0d4\bin' 'bc2'
1 --> chocolate given.
2 --> chocolate given.
3 --> chocolate given.
4 --> chocolate given.
5 --> chocolate given.
7 --> chocolate given.
9 --> chocolate given.
10 --> chocolate given.
PS F:\VS Code\CSE\Java\C
```

এখানে 6 & 8 কে chocolate দিবো না তাই continue অর্থাৎ 6 & 8 কে বাদ দিয়ে Loop এ ঘুরবে।

Nested Loops

Key points of the structure:

- We write a brand new loop inside the body of another loop.
- In the nested loop we need to make sure to use different variable than the one we used in the outer loop.
- For each iteration of the outer loop, the inner loop executes all its iteration.

Structure of **nested for loop**:

```
1.  for ( INITIALIZATION_1 ; CONDITION_1 ; STEPS_1 ) {
2.          for ( INITIALIZATION_2 ; CONDITION_2 ; STEPS_2 ) {
3.                  BODY OF THE LOOP
4.          }
5.  }
```

while loop দিয়ে –

```
3  public class nes1 {
       Run | Debug
4      public static void main(String [] args)
5      {
6          int outer = 1;
7          while(outer <= 2) // 1 <= 2
8          {
9              int inner = 1;
10             while (inner <= 3) // 1 <= 3
11             {
12                 System.out.println(outer + " " + inner); // 1 1
13                 inner += 1; // 1 + 1 = 2
14             }
15             System.out.println(x:"Inner Loop Shesh");
16             outer += 1; // 1 + 1 = 2
17         }
18     }
19 }
```

```
de\User\workspaceStor
t_ws\Loops_912140d4\b
1 1
1 2
1 3
Inner Loop Shesh
2 1
2 2
2 3
Inner Loop Shesh
PS F:\VS Code\CSE\Jav
```

for loop দিয়ে-

```
3  public class nes2 {
       Run | Debug
4      public static void main(String [] args)
5      {
6          for (int outer = 1; outer <= 2; outer++) // 1<=2; 1+1=2 --> 2<=2
7          {
8              for (int inner = 1; inner <= 3; inner++) // 1<=3; 1+1=2 --> 2<=3
9              {
10                 System.out.println(outer + " " + inner); // 1 1     1 2
11             }
12             System.out.println(x:"Inner Loop Shesh");
13         }
14     }
15 }
```

```
es  -cp  C:\User
spaceStorage\18a49
\jdt_ws\Loops_9121
1 1
1 2
1 3
Inner Loop Shesh
2 1
2 2
2 3
Inner Loop Shesh
PS F:\VS Code\CSE\
```

Practice Problem



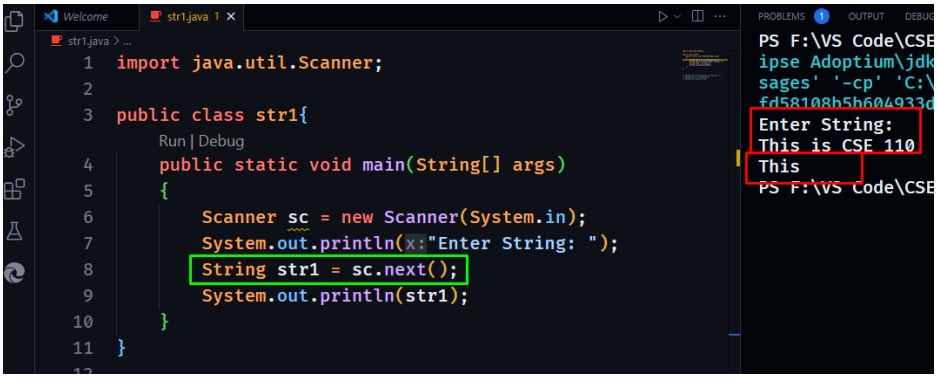
Strings

What is a string?

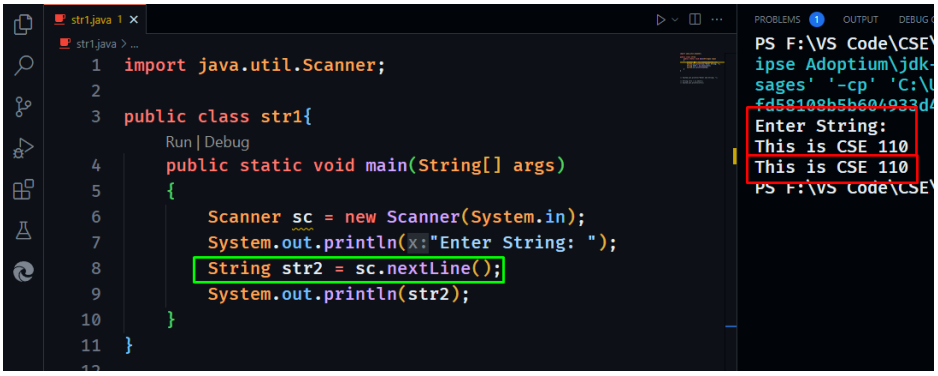
- In many other programming languages, string is a sequence of characters.
- In Java, strings are objects.
- Strings are surrounded by double quotes.
- Strings are immutable. Once a string is created, it can't be changed.
- If we want to modify a string, it will create a new string which contains the modifications.

String Input and Output

- .next() and .nextLine() of the scanner class is used to take string as input.
- .next() only takes input until it finds a " " or space in the given input.
- .nextLine() takes input until it finds a "\n" in the given input.



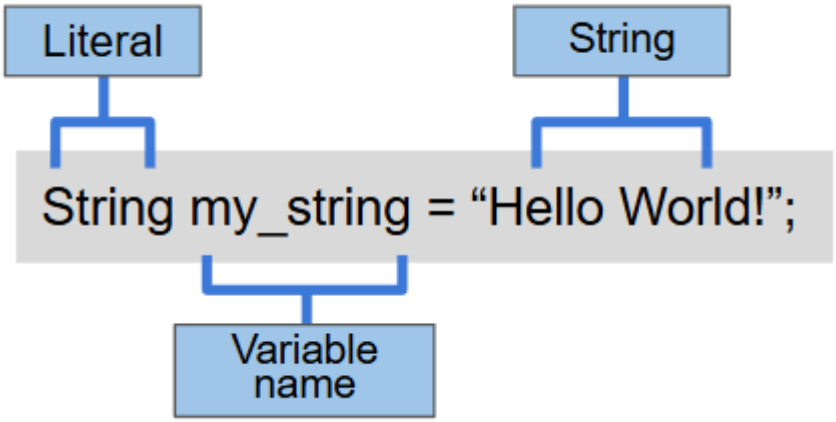
String Input নেয়ার সময় sc.next() এর ক্ষেত্রে প্রথম space পর্যন্ত নিবে।



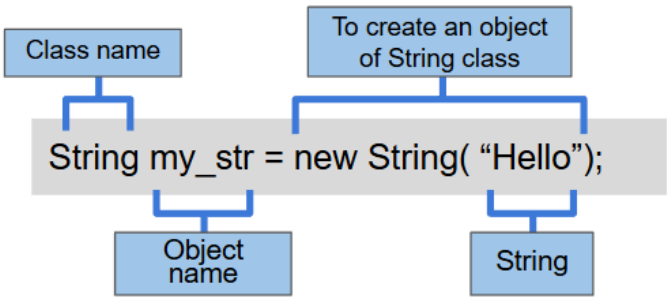
sc.nextline() এর ক্ষেত্রে সম্পূর্ণ লাইন নিবে।

String Initialization

- Initialization using String literals:



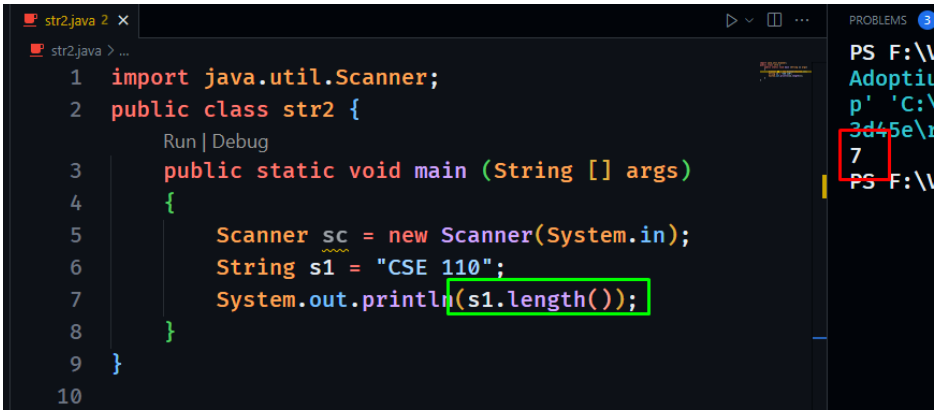
- Creating an object of String class:



String Length

- The number of characters in a string.
- Can find using the length() method.
- String indexing starts from 0 and ends at (Length of string - 1). [০ থেকে শুরু হলে মোট যতগুলো আছে তার থেকে ১টি কম হয়। যেমনঃ ১ টা String এ যদি ৭ টা character থাকে তাহলে তার মোট index হবে ০ থেকে ৬]
- If we create string1 = “CSE IS FUN.”, which has a length of 11, the indexing will look like this:

0	1	2	3	4	5	6	7	8	9	10
C	S	E		I	S		F	U	N	.

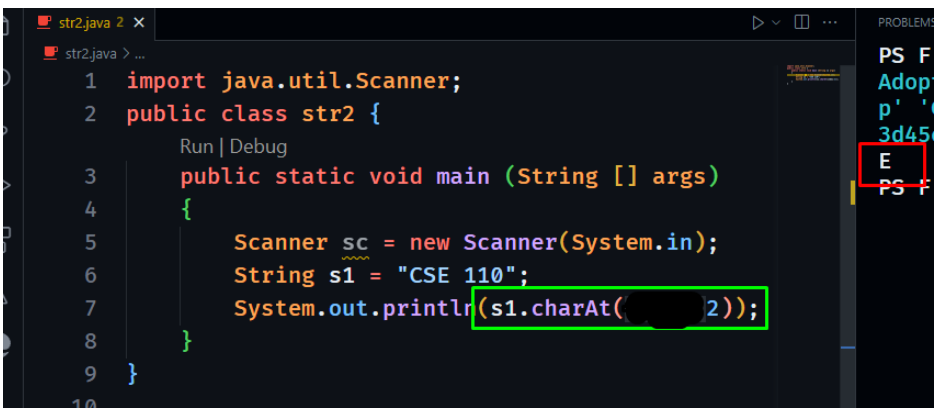


- Blanks and special characters are also counted as characters. (ফাঁকা জায়গা/Space ও ১ টা Character হিসেবে গণ্য হবে।)

.charAt(index) Method

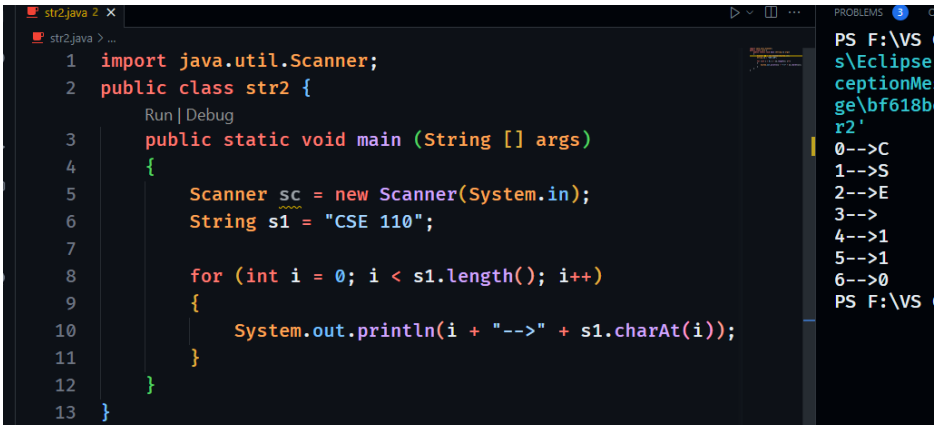
- .charAt(index)
  - Returns the character at index value passed as argument.

- We can get characters at each index by using **charAt(index)** method. Example: string1.charAt(2) will return “E”.



Iterating a String

- Accessing the characters of a string one by one.
- Iterating a string using for loop and charAt() method.



```java
import java.util.Scanner;
public class str2 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            System.out.println(i + "-->" + s1.charAt(i));
        }
    }
}
```

```
PS F:\VS
s\Eclipse
ceptionMe
ge\bf618b
r2'
0-->C
1-->S
2-->E
3-->
4-->1
5-->1
6-->0
PS F:\VS
```

এখানে যেই String নিয়েছি loop এর মাধ্যমে সেটার index অনুযায়ী character গুলো বের করে এনেছি।

আর এখানে **i < s1.length(),** কেননা ০ থেকে শুরু করে index, total length এর আগে পর্যন্ত যাবে।

ASCII values

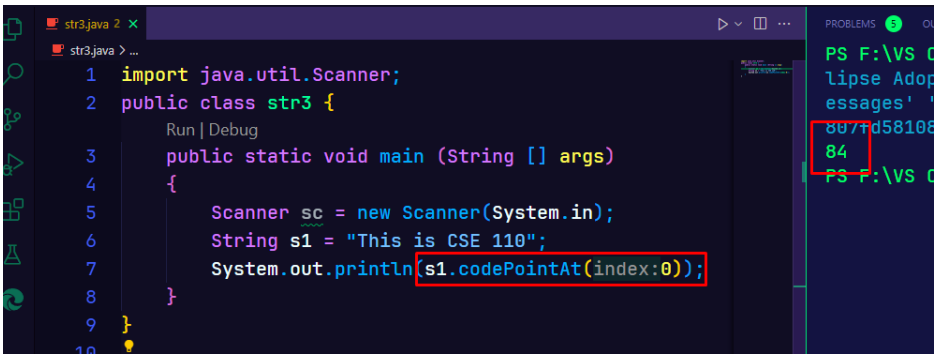
- Unique Unicode value assigned to all keyboard characters.
- Range of ASCII values of **uppercase characters ('A' to 'Z'):** 65 - 90
- Range of ASCII values of **lowercase characters ('a' to 'z'):** 97 - 122
- ASCII value of space: 32
- ASCII value of (0-9): 48 – 57

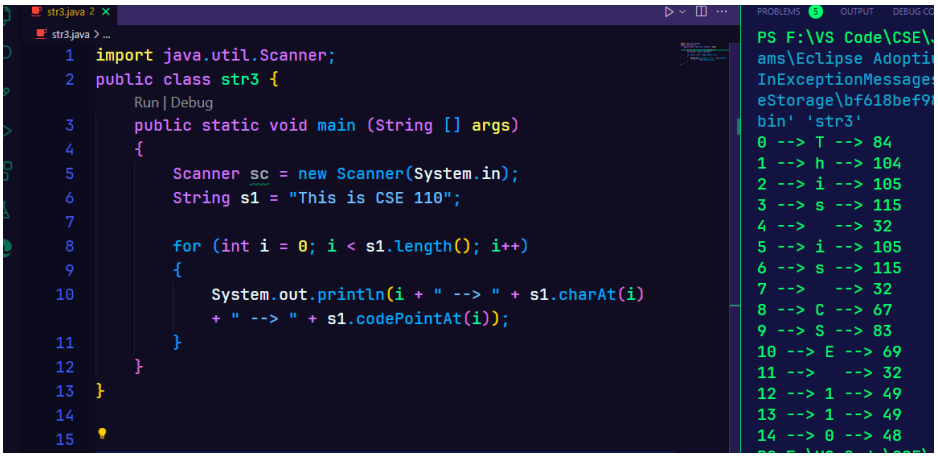
```
12
13 // A to Z - 65 to 90
14 // A - 65
15 // B - 66
16
17 // a to z - 97 to 122
18 // a - 97
19 // b - 98
20
21 // 'a' 97-32 = 65 'A'
22 // 'b' 98-32 = 66 'B'
23
24 // 'A' 65+32 = 97 'a'
25 // 'B' 66+32 = 98 'b'
26
27
```

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL (null)	32	SPACE	64	@	96	`
1	SOH (start of heading)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	TAB (horizontal tab)	41	)	73	I	105	i
10	LF (NL line feed, new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (NP form feed, new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (device control 1)	49	1	81	Q	113	q
18	DC2 (device control 2)	50	2	82	R	114	r
19	DC3 (device control 3)	51	3	83	S	115	s
20	DC4 (device control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of trans. block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	|
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL

Converting a Character to ASCII Value

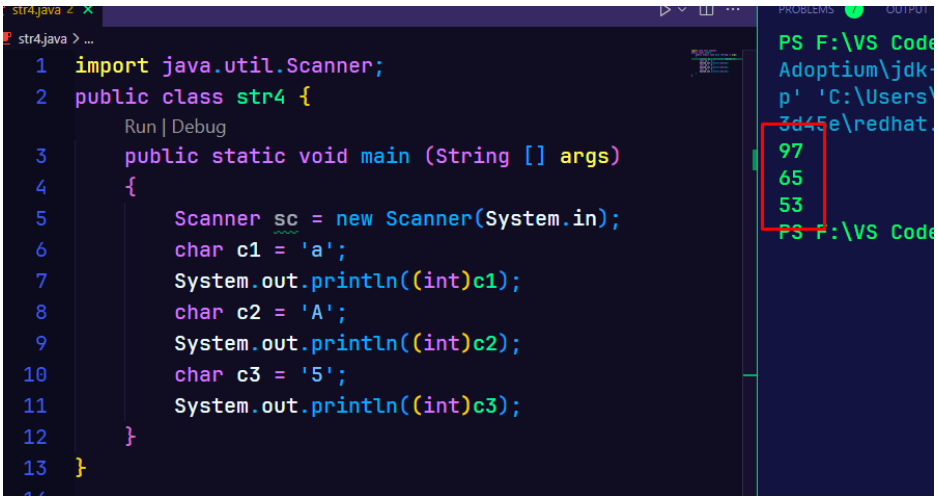
- .codePointAt(index)
  - Takes an index of a string as a parameter and returns the ASCII value of the character of that index.





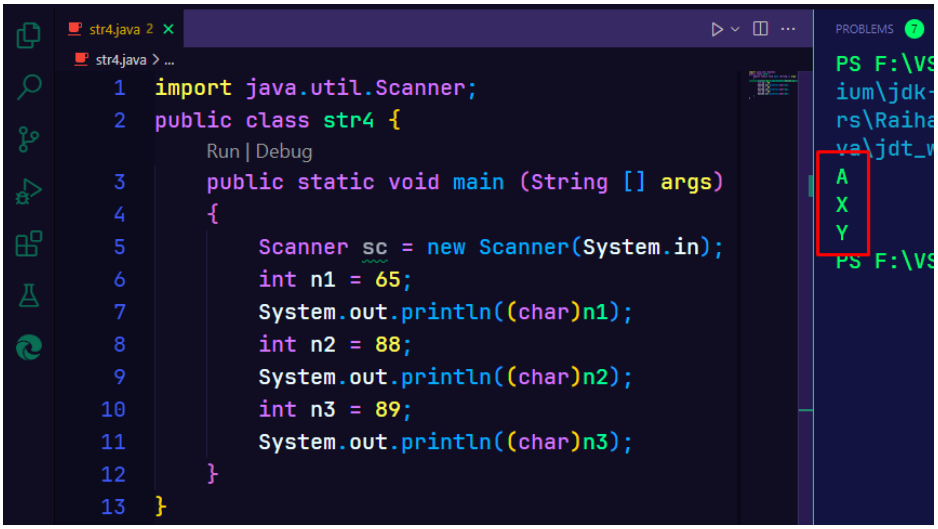
এখানে যেই String নিয়েছি loop এর মাধ্যমে সেটার index অনুযায়ী character এবং সেই character গুলোর ASCII value গুলো বের করে এনেছি।

- int(chr)
  - Can simply cast a character as an integer.



এটাকে casting বলে। মূলত character গুলোর ASCII value বের করে আনা।

আবার উল্টা কাজও করতে পারি অর্থাৎ number/ASCII value থাকলে তার character বের করে আনা।

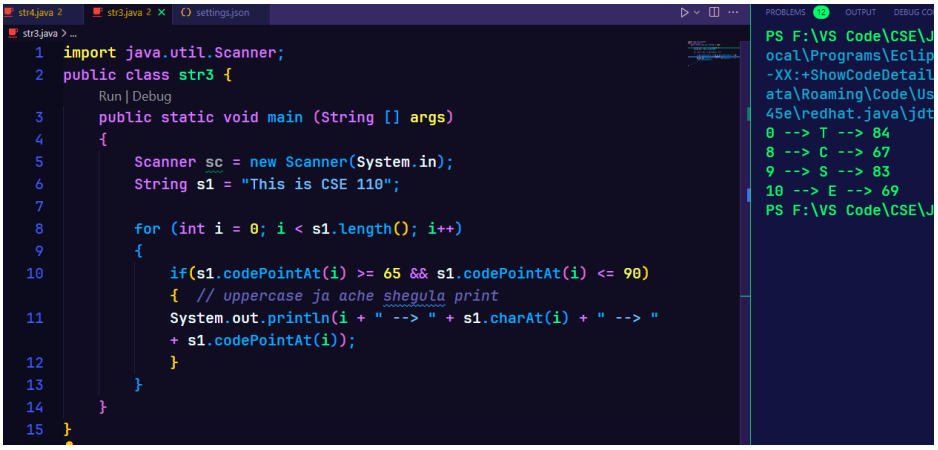




```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.codePointAt(i) >= 97 && s1.codePointAt(i) <= 122)
            {  // Lowercase ja ache shegula print
            System.out.println(i + " --> " + s1.charAt(i) + " --> "
            + s1.codePointAt(i));
            }
        }
    }
}
```

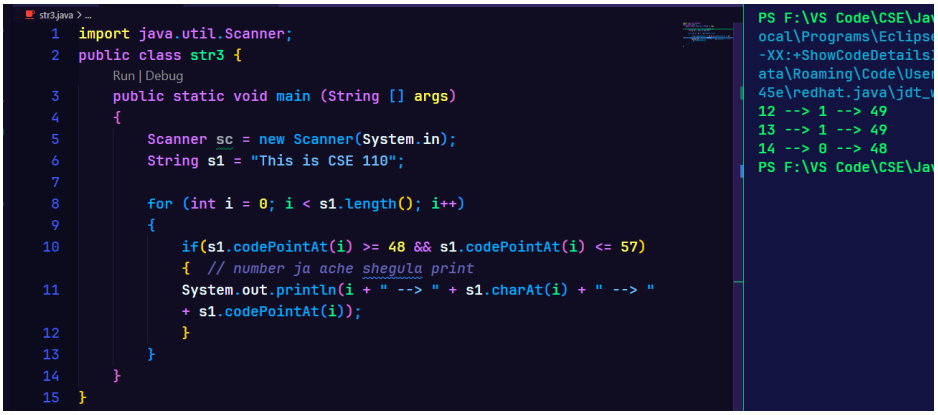
```
PS F:\VS Code\CSE\Java\CSE11
ocal\Programs\Eclipse Adopti
-XX:+ShowCodeDetailsInExcept
ata\Roaming\Code\User\works
45e\redhat.java\jdt_ws\Strin
1 --> h --> 104
2 --> i --> 105
3 --> s --> 115
5 --> i --> 105
6 --> s --> 115
PS F:\VS Code\CSE\Java\CSE11
```



```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.codePointAt(i) >= 65 && s1.codePointAt(i) <= 90)
            {  // uppercase ja ache shegula print
            System.out.println(i + " --> " + s1.charAt(i) + " --> "
            + s1.codePointAt(i));
            }
        }
    }
}
```

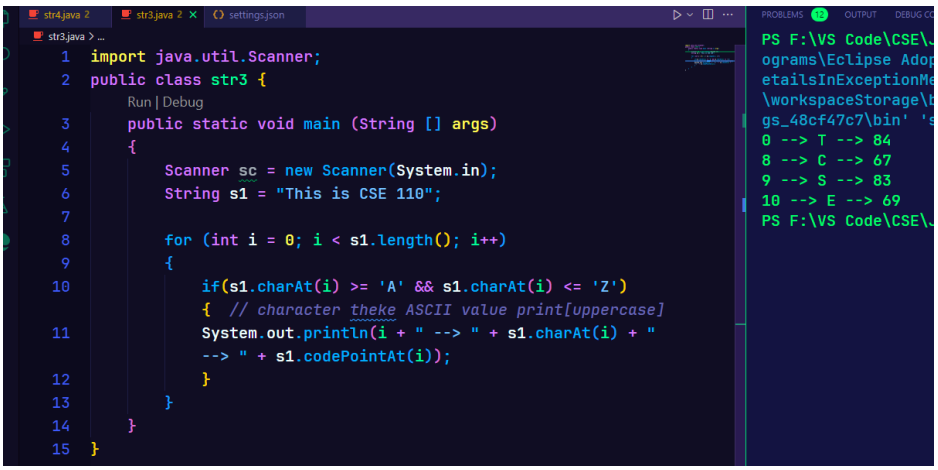
```
PS F:\VS Code\CSE\J
ocal\Programs\Eclip
-XX:+ShowCodeDetail
ata\Roaming\Code\Us
45e\redhat.java\jdt
0 --> T --> 84
8 --> C --> 67
9 --> S --> 83
10 --> E --> 69
PS F:\VS Code\CSE\J
```



```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.codePointAt(i) >= 48 && s1.codePointAt(i) <= 57)
            {  // number ja ache shegula print
            System.out.println(i + " --> " + s1.charAt(i) + " --> "
            + s1.codePointAt(i));
            }
        }
    }
}
```

```
PS F:\VS Code\CSE\Jav
ocal\Programs\Eclipse
-XX:+ShowCodeDetails
ata\Roaming\Code\User
45e\redhat.java\jdt_w
12 --> 1 --> 49
13 --> 1 --> 49
14 --> 0 --> 48
PS F:\VS Code\CSE\Jav
```



```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.charAt(i) >= 'A' && s1.charAt(i) <= 'Z')
            {  // character theke ASCII value print[uppercase]
            System.out.println(i + " --> " + s1.charAt(i) + "
            --> " + s1.codePointAt(i));
            }
        }
    }
}
```

```
PS F:\VS Code\CSE\J
ograms\Eclipse Adop
etailsInExceptionMe
\workspaceStorage\b
gs_48cf47c7\bin' 's
0 --> T --> 84
8 --> C --> 67
9 --> S --> 83
10 --> E --> 69
PS F:\VS Code\CSE\J
```

```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.charAt(i) >= 'a' && s1.charAt(i) <= 'z')
            {  // character theke ASCII value print[lowercase]
            System.out.println(i + " --> " + s1.charAt(i) + "
            --> " + s1.codePointAt(i));
            }
        }
    }
}
```

```
PS F:\VS Code\CSE
ograms\Eclipse Ad
etailsInException
\workspaceStorage
gs_48cf47c7\bin'
1 --> h --> 104
2 --> i --> 105
3 --> s --> 115
5 --> i --> 105
6 --> s --> 115
PS F:\VS Code\CSE
```

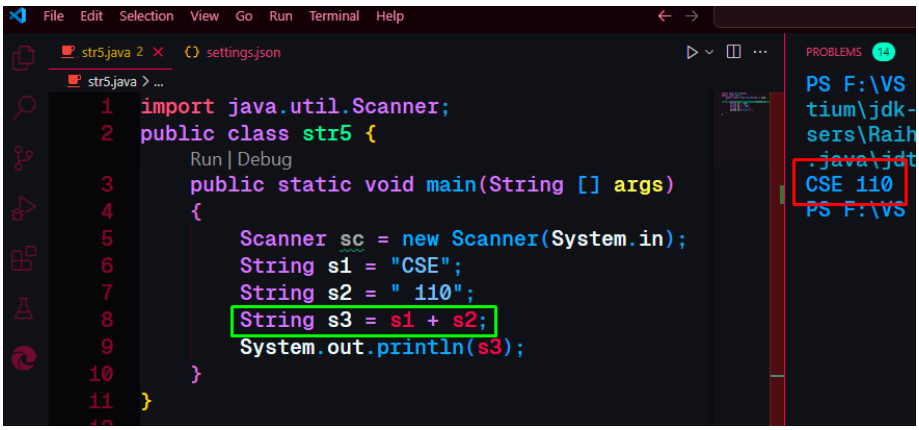
```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.charAt(i) >= '0' && s1.charAt(i) <= '9')
            {  // character theke ASCII value print[numbers]
            System.out.println(i + " --> " + s1.charAt(i) + "
            --> " + s1.codePointAt(i));
            }
        }
    }
}
```

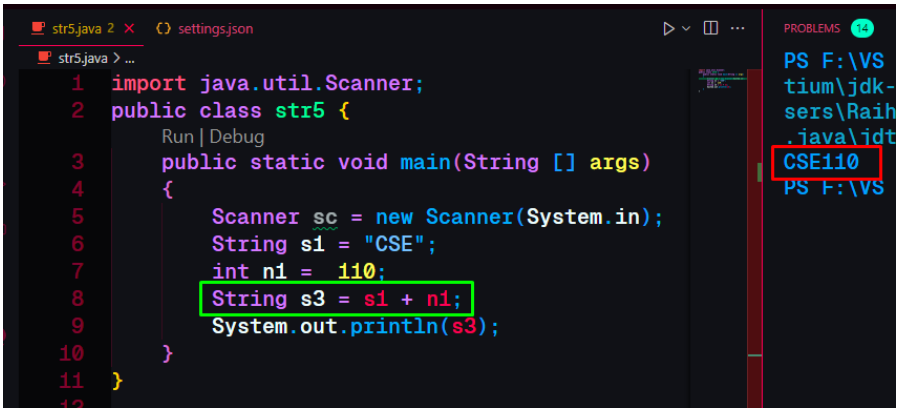
```
ograms\Eclipse Ad
etailsInExceptionM
\workspaceStorage\
gs_48cf47c7\bin'
12 --> 1 --> 49
13 --> 1 --> 49
14 --> 0 --> 48
PS F:\VS Code\CSE\
```

String Concatenation

- "+" is used to concatenate two or more strings together.
- New string created after concatenation.
- If a different data type follows a string data type in concatenation, that data is automatically converted to string.

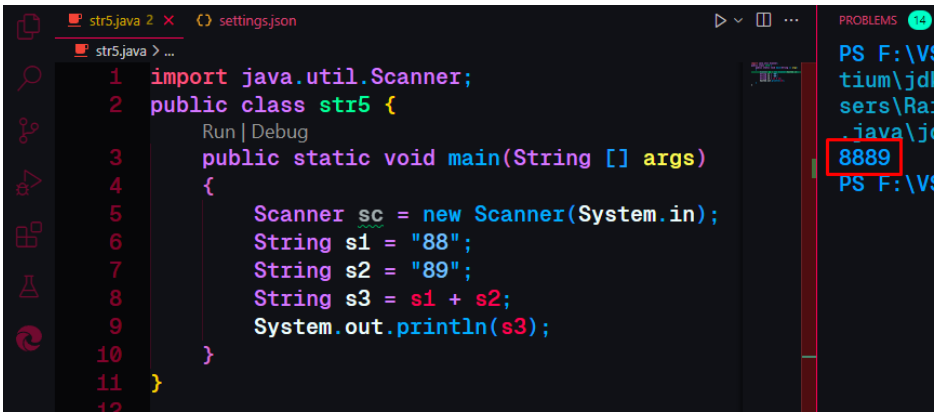


- Number + Number = Addition
- “String” + Number = Concatenation



- “String” + “String” = String

অর্থাৎ, ২ টা জিনিস (“ ”) এর ভিতরে/String হিসেবে থাকলে Attach হয়ে যাবে।



Comparing Strings

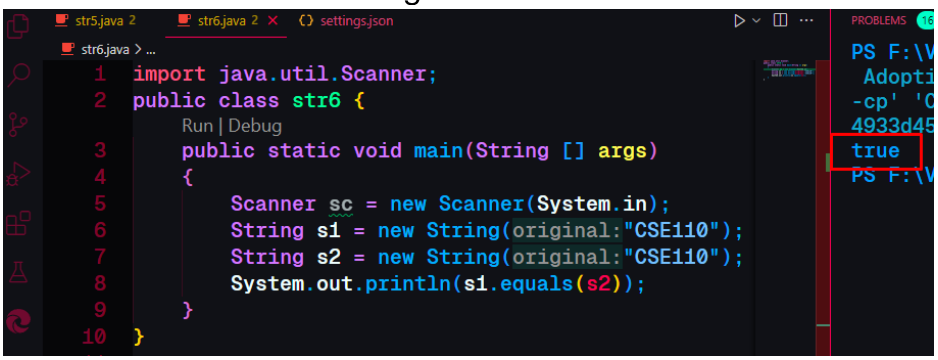
- String str1 = new String(“Hello”); String str2 = new String(“hello”);

Methods	Functions	Method Call	Value returned
.equals()	Checks if values in two strings are same. Case sensitive.	str1.equals(str2)	False
.equalsIgnoreCase()	Checks if values in two strings are same. Not case sensitive.	str1.equalsIgnoreCase(str2)	True
== or equals operator	Checks if reference (address) of two strings are same.	str1 == str2	False

.compareTo()	Compares two string values lexicographically. Returns 0 when identical, (+)ve when first string is greater and (-)ve values when second string is greater respectively.	str1.compareTo(str2)	-32 (The output is -32 since “H” is 32 less than “h” lexicograp hically)

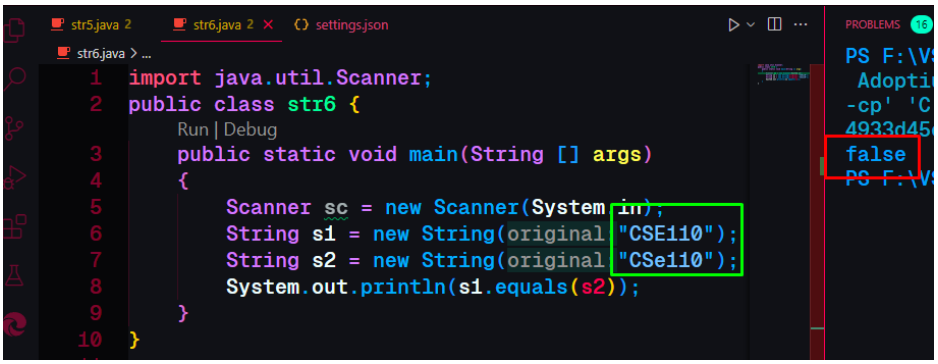
.equals( )

Checks if values in two strings are same. Case sensitive.



```java
import java.util.Scanner;
public class str6 {
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = new String(original:"CSE110");
        String s2 = new String(original:"CSE110");
        System.out.println(s1.equals(s2));
    }
}
```

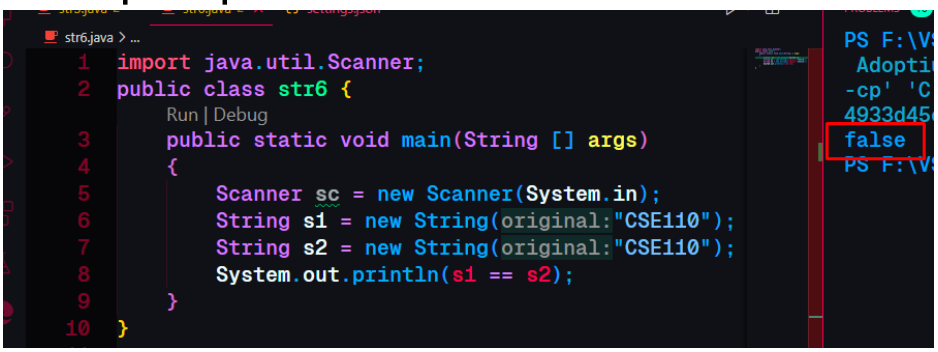
```
true
```



```java
import java.util.Scanner;
public class str6 {
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = new String(original "CSE110");
        String s2 = new String(original "CSe110");
        System.out.println(s1.equals(s2));
    }
}
```

```
false
```

== or equals operator



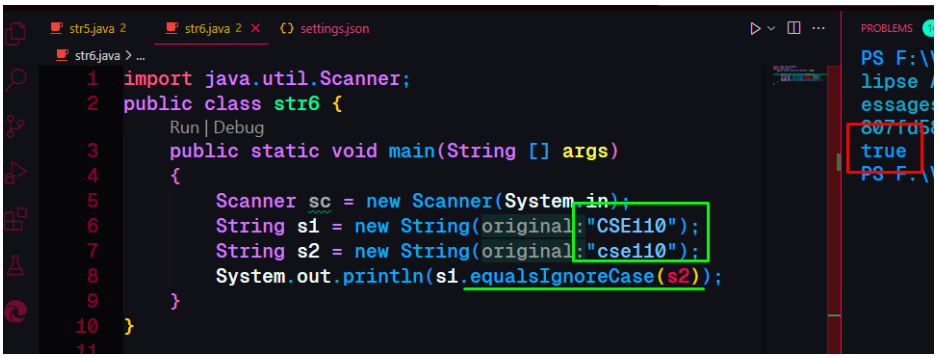
```java
import java.util.Scanner;
public class str6 {
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = new String(original:"CSE110");
        String s2 = new String(original:"CSE110");
        System.out.println(s1 == s2);
    }
}
```

```
false
```

== সবসময় Reference(Address) যাচাই করে।

.equalsIgnoreCase( )

Checks if values in two strings are same. Not case sensitive.

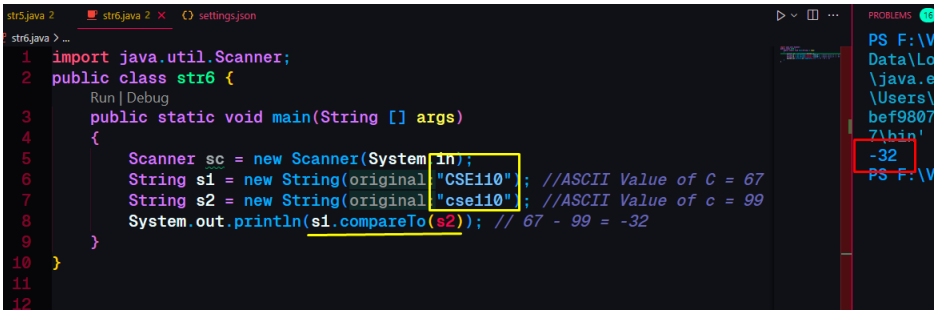


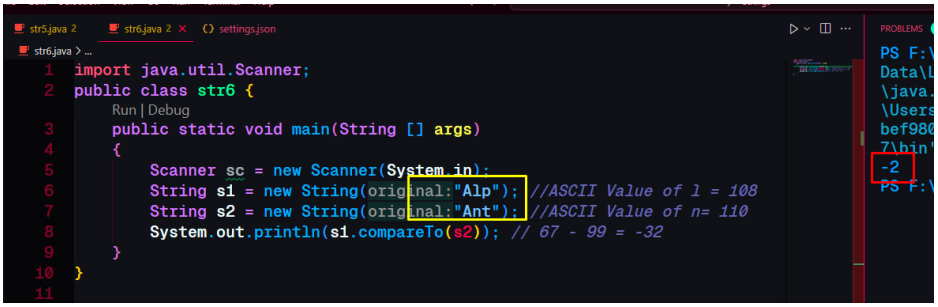
```java
import java.util.Scanner;
public class str6 {
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = new String(original:"CSE110");
        String s2 = new String(original:"cse110");
        System.out.println(s1.equalsIgnoreCase(s2));
    }
}
```

```
true
```

.compareTo( )

Compares two string values lexicographically. Returns 0 when identical, (+)ve when first string is greater and (-)ve values when second string is greater respectively.

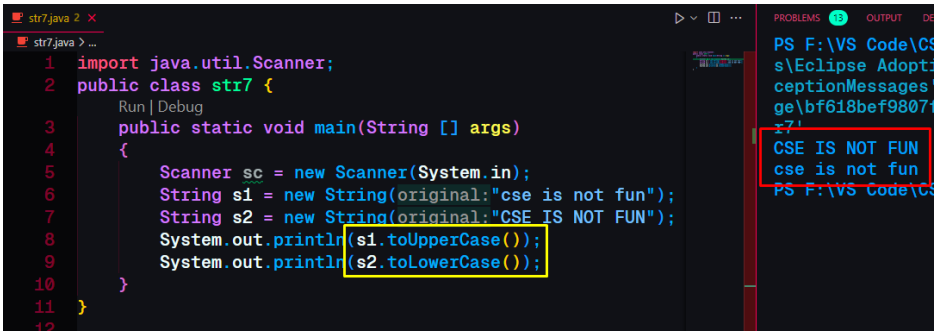




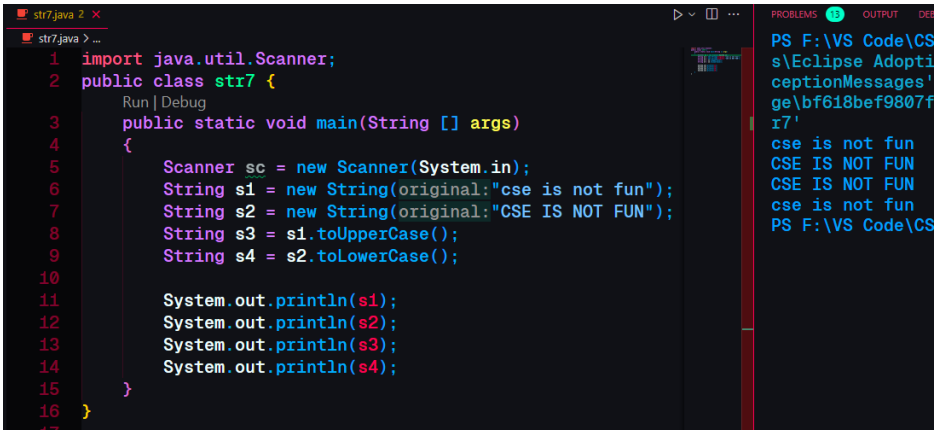
প্রথম অমিল যেখানে পাচ্ছে সেখানে পরস্পর ২ টি character এর ASCII value compare বা পার্থক্য বের করছে।

Case Conversion

- .toLowerCase( )
  - Converts all characters to lowercase letters.
- .toUpperCase( )
  - Converts all characters to uppercase letters.



String কে কখনোই আমরা change করতে পারি না। String কে change করতে হলে আবার নতুন করে String তৈরি করতে হবে।



এখানে, s1 & s2 পরিবর্তন হয়নি। s1, s2 এর ১ টা UpperCase & LowerCase version তৈরি করে s3 & s4 এ store করে দিয়েছে।

Arrays

What is an array?

- An array is a collection of values of the same data type.
- The length of an array remains constant once it's created.
- Multiple elements are separated using comma when initialized.
- Arrays are considered as objects in java.
- Number of elements of an array is called the length of an array.

◆ **An Array is an Indexed collection of, fixed number of Homogeneous Data elements.**

◆ **Homogeneous** বলতে একই রকম Data type বোঝায়। যদি int [] array নিয়ে value -> array[2] = 4.44 অর্থাৎ double value নেয়া হয় তাহলে error দিবে।

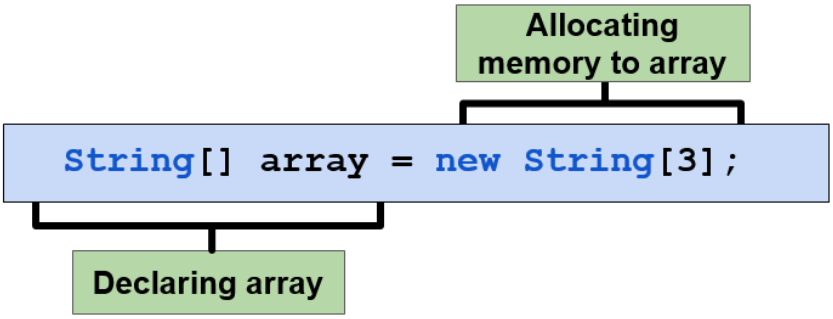
◆ We can represent huge number of values by using single variable so that Readability of the code will be improved. (একটা Variable এ অনেক বেশি value নিতে পারি যার ফলে Code পড়তে/বুঝতে সুবিধা হয়।)

◆ But the main disadvantage of arrays is fixed in size. Once we create an array, there is no chance of increasing/decreasing the size based on our requirement. (প্রধান অসুবিধা হলো Array এর সাইজ কিংবা আকার সুনির্দিষ্ট। একবার ১ টি Array তৈরি করা হয়ে গেলে কোনোভাবেই আর আমাদের প্রয়োজন অনুসারে Array এর আয়তন বাড়াতে/কমাতে পারবো না।)

◆ Hence, To use arrays concept, compulsory we should know the size in advance, which may not possible always. (Array এর ব্যাপারগুলো ব্যবহার করতে হলে Array এর আয়তন আগে থেকেই বাধ্যতামূলকভাবে জানা উচিত, যেটা সবসময় সম্ভব নাও হতে পারে।)

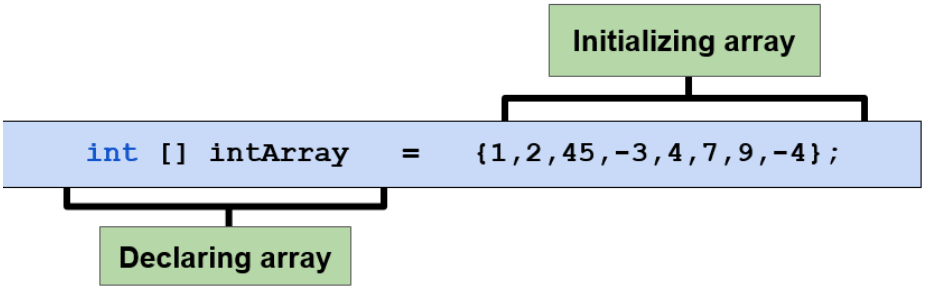
Array Declaration

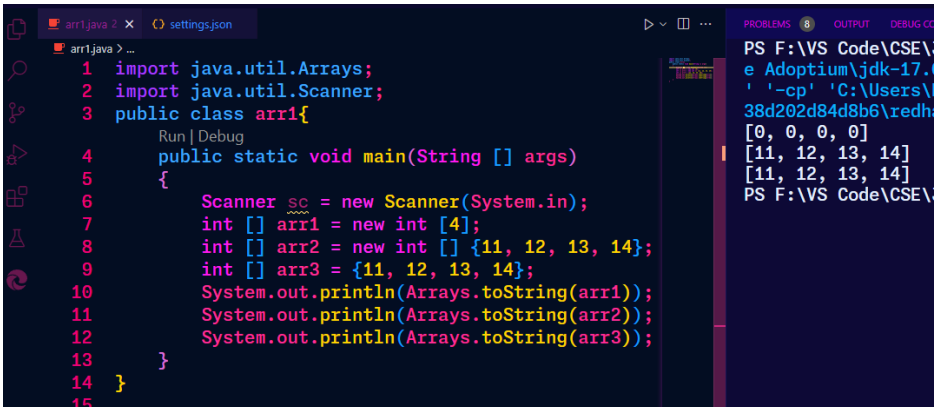
- “new” Keyword is used to create arrays.
- While initializing an array we declare the type of the elements and the size of the array
- Initially array contains the default values of the specified data type.
- Syntax:



Array Initialization

- We can initialize an array at the time of declaration.
- Initializing using an array initializer (a comma-separated list of expressions enclosed in braces)
- Syntax:





```java
import java.util.Arrays;
import java.util.Scanner;
public class arr1{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int [] arr1 = new int [4];
        int [] arr2 = new int [] {11, 12, 13, 14};
        int [] arr3 = {11, 12, 13, 14};
        System.out.println(Arrays.toString(arr1));
        System.out.println(Arrays.toString(arr2));
        System.out.println(Arrays.toString(arr3));
    }
}
```

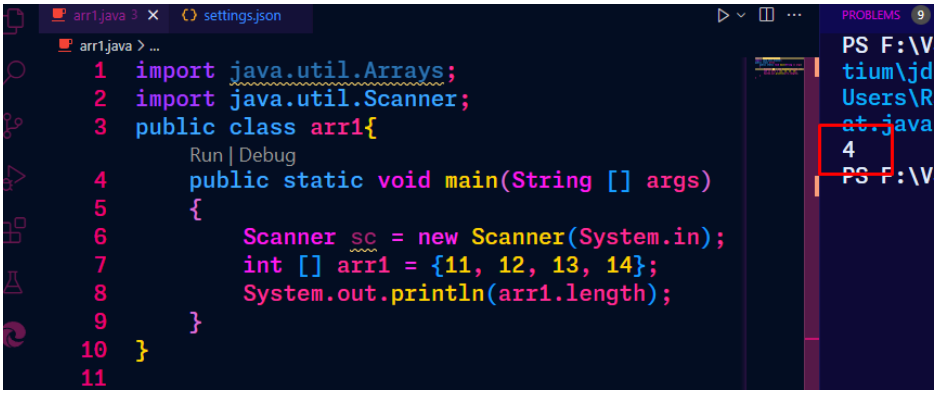
```
PS F:\VS Code\CSE\
e Adoptium\jdk-17.
' '-cp' 'C:\Users\
38d202d84d8b6\redh
[0, 0, 0, 0]
[11, 12, 13, 14]
[11, 12, 13, 14]
PS F:\VS Code\CSE\
```

Index of an Array

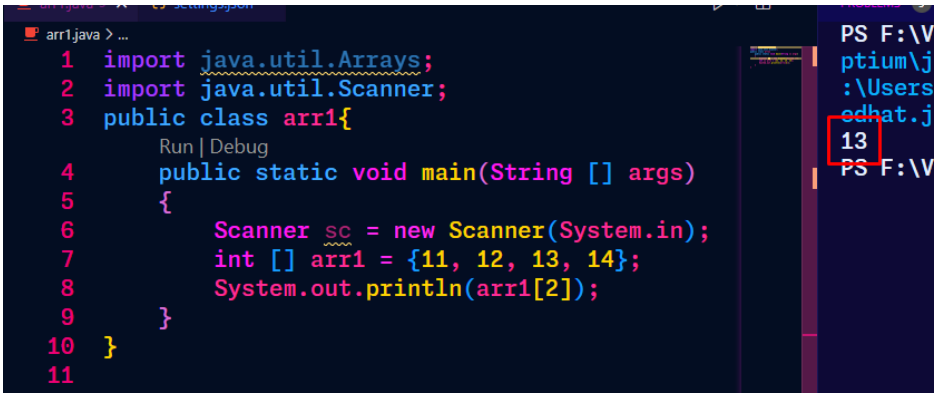
- Each elements of an array holds a position or index.
- The first element in every array has index **zero**.
- The last element has index number (**array.length - 1)**
- Index number is enclosed by square brackets [ ]

Elements	44	3	1	12	-10	38	0	55
Indices	0	1	2	3	4	5	6	7

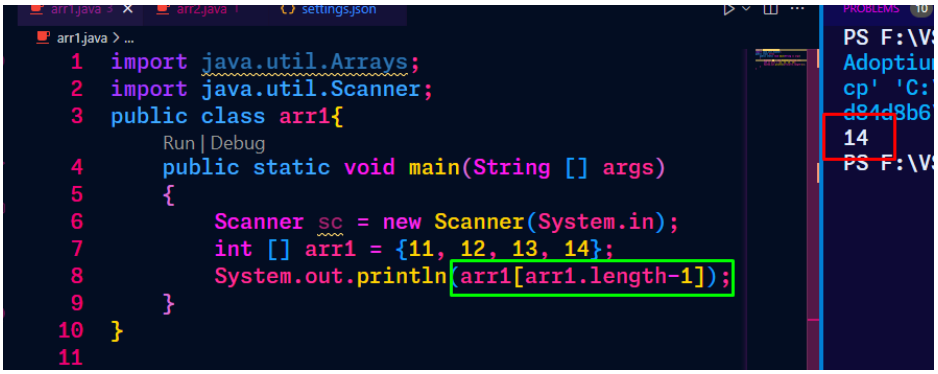
Length = 8



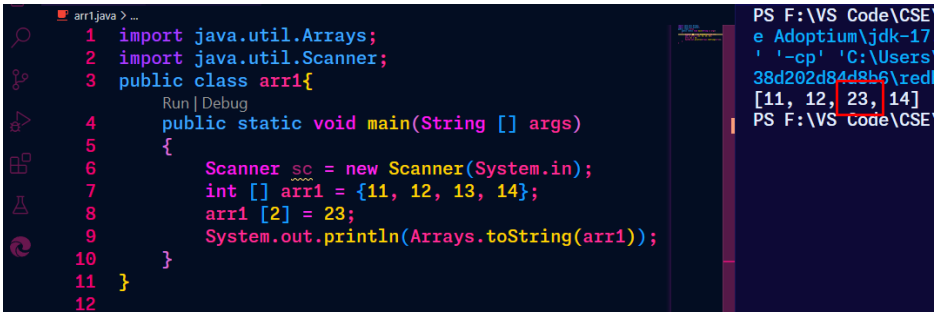
Array এর ক্ষেত্রে method হচ্ছে .length। .length( ) নয়।



২ নং index এ যেটা আছে সেটা print করা। এভাবে যেকোনো index এর value print করা যায়।



Array এর শেষ value print করতে চাইলে index অনুযায়ী arr1[ ] দিয়ে করা যায়। একই সাথে এভাবেও করা যায়।

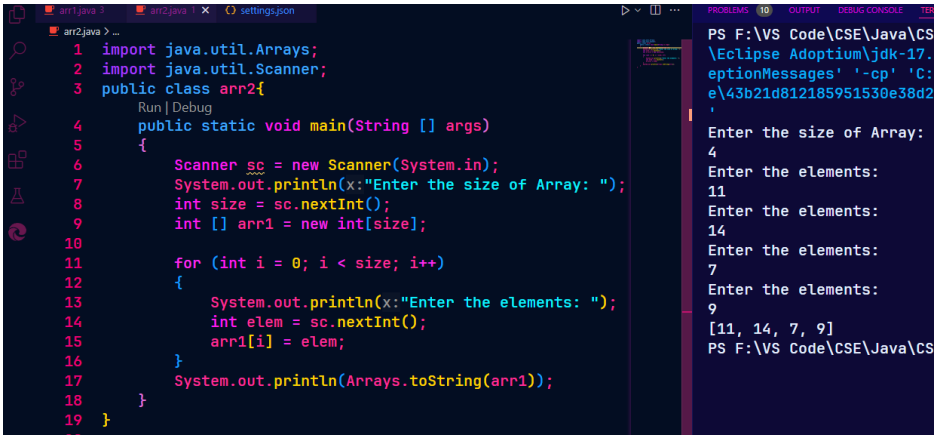


```java
import java.util.Arrays;
import java.util.Scanner;
public class arr1{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int [] arr1 = {11, 12, 13, 14};
        arr1 [2] = 23;
        System.out.println(Arrays.toString(arr1));
    }
}
```

```
PS F:\VS Code\CSE
e Adoptium\jdk-17
' '-cp' 'C:\Users
38d202d84d8b6\red
[11, 12, 23, 14]
PS F:\VS Code\CSE
```

Array এর value পরিবর্তন করা।

User থেকে input নিয়ে array তৈরি করাঃ



```java
import java.util.Arrays;
import java.util.Scanner;
public class arr2{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Enter the size of Array: ");
        int size = sc.nextInt();
        int [] arr1 = new int[size];

        for (int i = 0; i < size; i++)
        {
            System.out.println(x:"Enter the elements: ");
            int elem = sc.nextInt();
            arr1[i] = elem;
        }
        System.out.println(Arrays.toString(arr1));
    }
}
```

```
PS F:\VS Code\CSE\Java\CS
\Eclipse Adoptium\jdk-17.
eptionMessages' '-cp' 'C:
e\43b21d812185951530e38d2
'
Enter the size of Array:
4
Enter the elements:
11
Enter the elements:
14
Enter the elements:
7
Enter the elements:
9
[11, 14, 7, 9]
PS F:\VS Code\CSE\Java\CS
```

Array Iteration

- Array iteration means accessing each element of an array one by one.
- We can use the concept of loops to iterate an array.

Array Iteration - while loop

```java
import java.util.Arrays;
import java.util.Scanner;

public class arr3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int [] arr1 = {11, 12, 13, 14};
        System.out.println(x:"Elements of the Array are: ");

        int i = 0;
        while(i < arr1.length)
        {
            System.out.println(arr1[i]);
            i++;
        }
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE1
ams\Eclipse Adoptium\jdk-17
sInExceptionMessages' '-cp'
aceStorage\43b21d8121859515
5\bin' 'arr3'
Elements of the Array are:
11
12
13
14
PS F:\VS Code\CSE\Java\CSE1
```

Array Iteration - for loop

```java
arr4.java > ...
import java.util.Arrays;
import java.util.Scanner;

public class arr4 {
    Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int[] arr1 = { 11, 12, 13, 14 };
        System.out.println(x:"Elements of the Array are: ");
        for (int i = 0; i < arr1.length; i++)
        {
            System.out.println(arr1[i]);
        }
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE1
ams\Eclipse Adoptium\jdk-17
sInExceptionMessages' '-cp'
aceStorage\43b21d8121859515
5\bin' 'arr4'
Elements of the Array are:
11
12
13
14
PS F:\VS Code\CSE\Java\CSE1
```

একই array কে উল্টা (বিপরীত দিক থেকে) print করতে চাইলেঃ

```java
arr5.java > ...
import java.util.Arrays;
import java.util.Scanner;

public class arr5 {
    Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int[] arr1 = { 11, 12, 13, 14 };
        System.out.println(x:"Elements of the Array are: ");
        for (int i = arr1.length-1; i >= 0; i--)
        {
            System.out.println(arr1[i]);
        }
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE1
ms\Eclipse Adoptium\jdk-17.
nExceptionMessages' '-cp' '
Storage\43b21d812185951530e
n' 'arr5'
Elements of the Array are:
14
13
12
11
PS F:\VS Code\CSE\Java\CSE1
```



Method/Function

Introduction to Methods

- ➢ Reusable blocks of code
- ➢ Perform specific tasks
- ➢ Can be called from any part of the code
- ➢ Consists of input parameters and return values
- ➢ Enhances code reusability, modularity, and maintainability
- ➢ Also known as functions

Types of Methods

1. Pre-defined/ built-in method:

These methods are already defined in java libraries e.g: .length(), .equals(), .compareTo(), .toLowerCase() etc.

2. User-defined method:

These are the methods written by the programmer.

User-Defined Methods

There are mainly 2 parts of using a user-defined method:

1. Method declaration
2. Method calling

Method Declaration can be further broken down into more parts.

Static Method

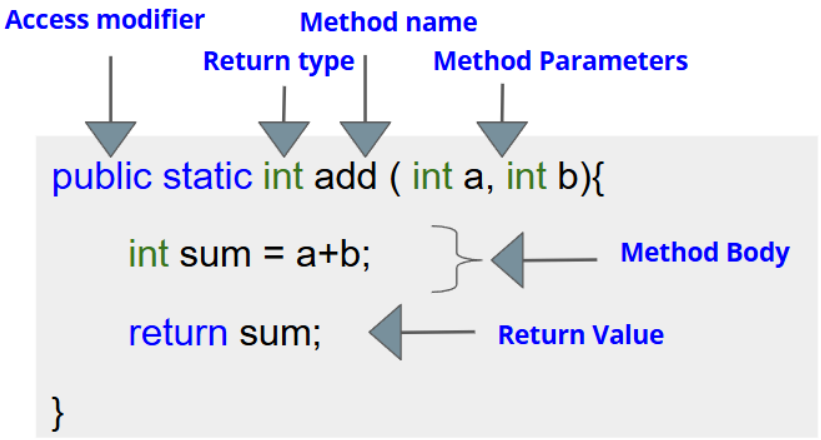
- A static method belongs to the class and not to the instance of a class.
- As it belongs to the class, it can be called without creating any object.
- We use the keyword “static” to define a method as static.
- We can call a static method from inside any other static method.

Method Declaration

- ➢ A method in java is written outside the main() method but inside the class.
- ➢ The following convention is followed when declaring a method:

```
access_modifier static return_type method_name (parameter_type parameter_name) {
        method body;
        return return_value;
}
```

Declaring a method that adds two numbers received as parameters.



Method Body

- Any set of code lines written inside the method.
- Any variable declared inside the scope of a method is not accessible from outside including the parameter variables.
- No line is executed after the “return” statement.

Return Type

- To declare a method we need to define what type of variable the method is returning.
- If we do not explicitly return any data or variable from the method it returns Null and that is declared as “void” in the method declaration.

1 void return

```
static void print_sum( int a, int b) {
    int sum = a+b;
    System.out.println(sum);
 } //this method does not return any value.
```

2 int return

```
static int sum( int a, int b) {
    int sum = a+b;
    return sum;
 } //this method returns an integer type value.
```

3 boolean return

```
static boolean check( int a) {
    if(a>10){
        return true;
    }
    else{
        return false;
    }
 } //this method returns a boolean type value.
```

4 String return

```
static String okay(int a, String str1) {
    return (a+2)+str1;
 } //this method returns a String type value.
```

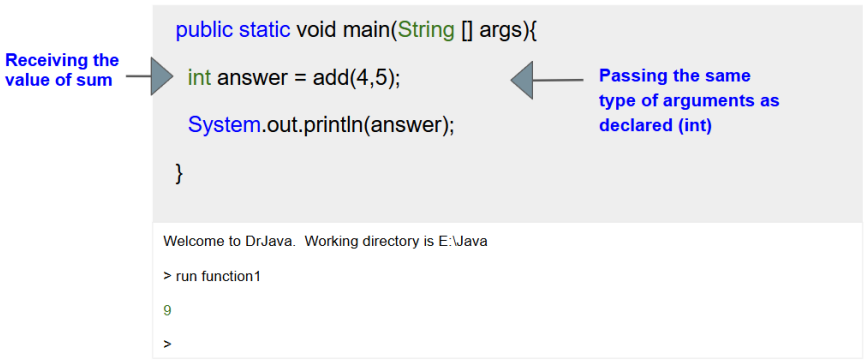
5 double return

```
static double point(int a) {
        double db = a/10.0;
        return db;
 } //this method returns a double type variable.
```

Method Call

- To run a method we MUST call the method.
- Method call is done inside the main() method.
- The number of arguments passed must be equal to parameters received.
- The sequence of the parameters passed is important.
- The type of the arguments passed must be same as type of parameters being received.
- After method call, a value is returned to the line of method call.

Calling the previously written add method.



Method Arguments & Parameters

The sequence of the arguments passed corresponds to the variables receiving the parameters.

Method to subtract two numbers

```
static void subtract( int a, int b) {
   int diff = a-b;
   System.out.println(diff);
 } //this method does not return any value.
```

Calling subtract

```
subtract(2, 3);
subtract(3,2);
```

Output

```
Welcome to DrJava.  Working directory is E:\Java
> run function1
-1
1
>
```

Here in case 1: subtract(2,3), the values 2 and 3 will be received as int a = 2 and int b = 3. Therefore, the difference is (2-3) = -1.

In case 2: int a = 3 and int b = 2 and so difference is (3-2) = 1.

Method Signature

- A method is identified based on its name, parameter list it receives and return type.
- Multiple methods can have same name but different sets of parameters.
- Multiple methods can have same name and same set of parameters but have different return types.
- The number of parameters passed in method call must be equal to the number of parameters received during method declaration.

Method Example 1

Writing a method that prints "Hello!"

```
public class func1{
  public static void sayHello(){
    System.out.println("Hello!");
  }
  public static void main(String []args){
    sayHello();
  }
}
```

Void return type as no value is being returned from inside

No arguments and no parameters received

Output

```
Welcome to DrJava.  Working directory is E:\Java
> run func1
Hello!
>
```

Method Example 2

Writing a method that checks if a number divisible by three or not.

```
public class func1{
  public static String divThree(int num){
    if(num%3==0){
      return "Divisible by 3";
    }
    else{
      return "Not divisible by 3";
    }
  }
  public static void main(String []args){
    String answer = divThree(13);
    System.out.println(answer);
  }
}
```

This value will be returned to the line of method call

13 will be received by num. And num = 13 will be accessible inside the function.

This line will invoke a method call.

Output

```
Welcome to DrJava.  Working directory is E:\Java
> run func1
Not divisible by 3
>
```

16

Method Example 3

Writing a method that prints the largest number out of two numbers.

```
public class func1{
  public static void findLargest(int num1, int num2){
    if(num1>num2){
      System.out.println(num1);
    }
    else{
      System.out.println(num2);
    }
  }
  public static void main(String []args){
    findLargest(13, -10);
  }
}
```

num1 = 13 num2 = -10

void return since no return statement.

Output

```
Welcome to DrJava.  Working directory is E:\Java
> run func1
13
>
```

17

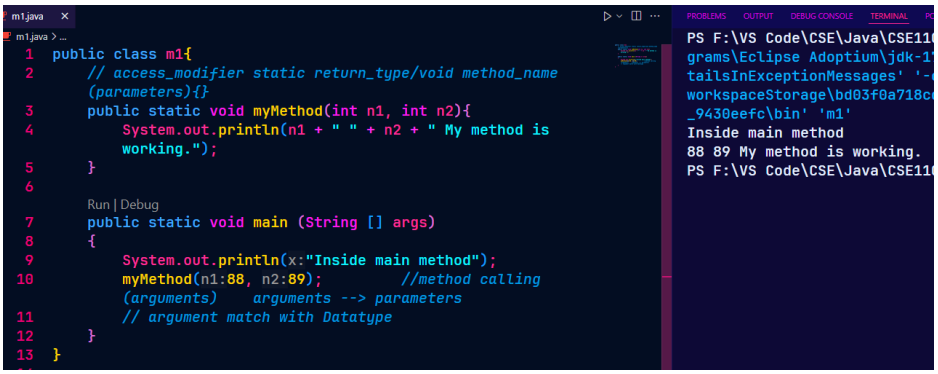
Calling a Method from another Method

- We can call a method from inside another method.

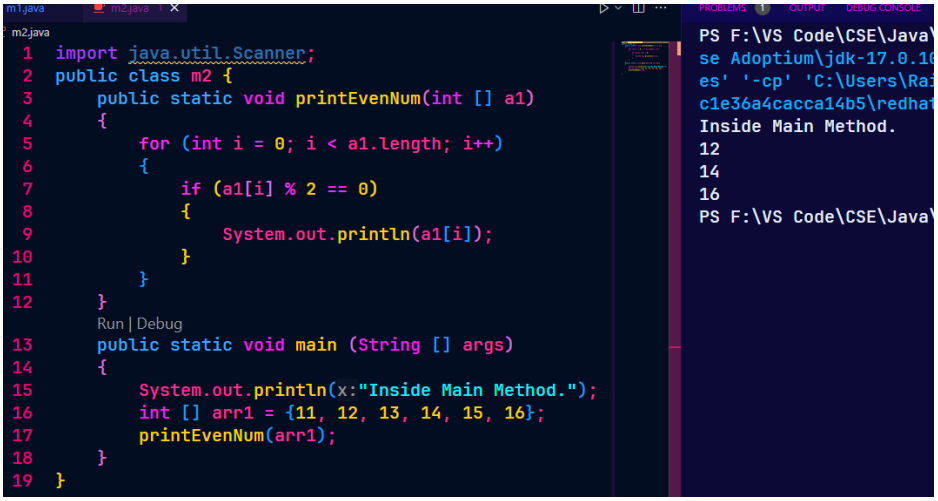
1. method1( ) called from the main( ) method.
2. method2( ) called from method1( ).

```
public class Methods {
    public static void main(String[] args) {
        method1();
    }
    static void method1(){
        System.out.print("CSE");
        method2();
    }
    static void method2(){
        System.out.print("110");
    }
}
```

```
Welcome to DrJava. Working directory is
C:\Users\Desktop
CSE110
```

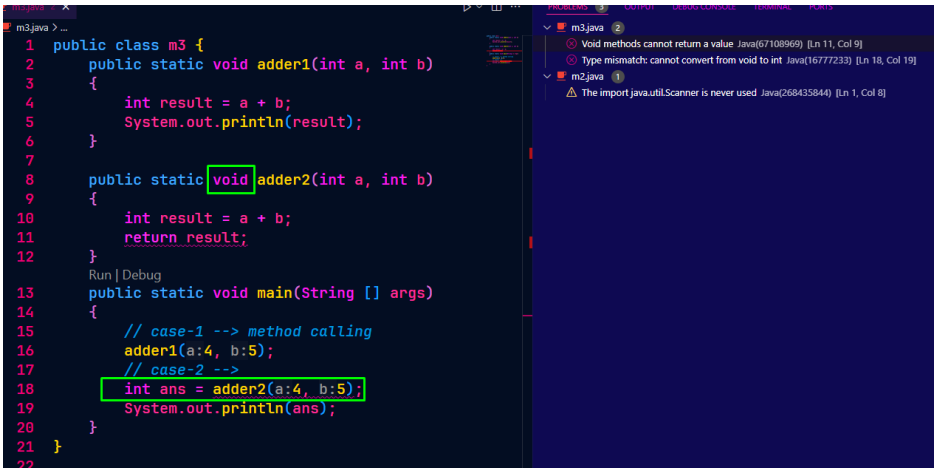


Code সবসময় main method থেকে [line-9] শুরু হবে। আগে ৯ নম্বর লাইনে যাবে অর্থাৎ myMethod কে call করবে এবং একই সাথে main method [line-9] এ যা দিচ্ছি call করা method এ [line-3] সেটার Data type টা অবশ্যই উল্লেখ করে দিতে হবে।



```java
import java.util.Scanner;
public class m2 {
    public static void printEvenNum(int [] a1)
    {
        for (int i = 0; i < a1.length; i++)
        {
            if (a1[i] % 2 == 0)
            {
                System.out.println(a1[i]);
            }
        }
    }
    Run | Debug
    public static void main (String [] args)
    {
        System.out.println(x:"Inside Main Method.");
        int [] arr1 = {11, 12, 13, 14, 15, 16};
        printEvenNum(arr1);
    }
}
```

```
PS F:\VS Code\CSE\Java\
se Adoptium\jdk-17.0.1
es' '-cp' 'C:\Users\Ra
c1e36a4cacca14b5\redha
Inside Main Method.
12
14
16
PS F:\VS Code\CSE\Java\
```

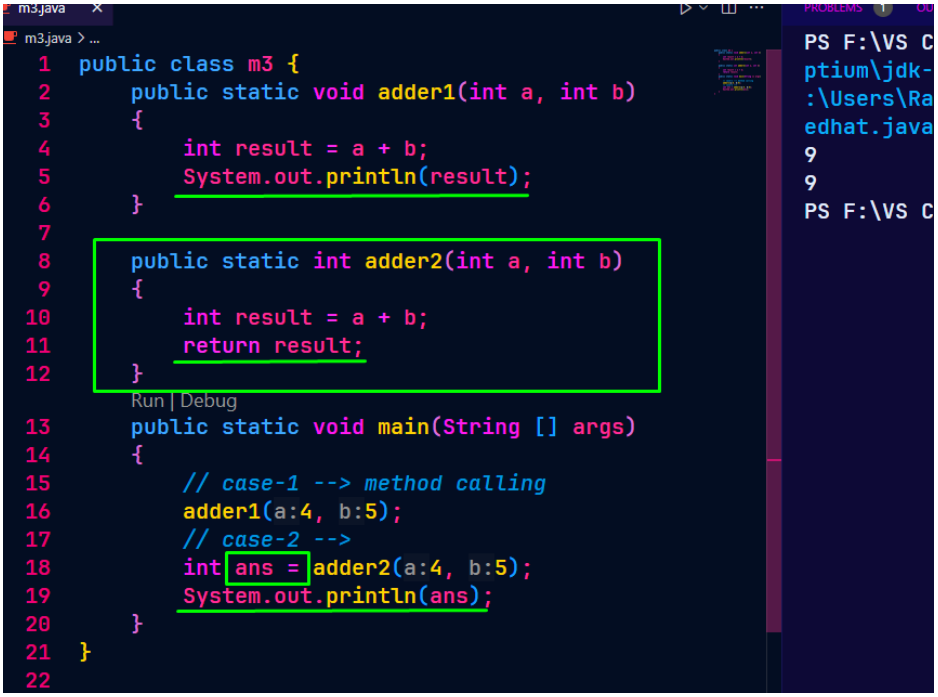


```java
public class m3 {
    public static void adder1(int a, int b)
    {
        int result = a + b;
        System.out.println(result);
    }

    public static void adder2(int a, int b)
    {
        int result = a + b;
        return result;
    }
    Run | Debug
    public static void main(String [] args)
    {
        // case-1 --> method calling
        adder1(a:4, b:5);
        // case-2 -->
        int ans = adder2(a:4, b:5);
        System.out.println(ans);
    }
}
```

```
m3.java
Void methods cannot return a value Java(67108969) [Ln 11, Col 9]
Type mismatch: cannot convert from void to int Java(16777233) [Ln 18, Col 19]
m2.java
The import java.util.Scanner is never used Java(268435844) [Ln 1, Col 8]
```

এখানে int ans = ১ টা output চাচ্ছি, চাই না adder2 method টা নিজে print করুক এবং সেটা void অর্থাৎ, এটা কোনোকিছুই ফেরত দেয় না/return করে না [Exit Gate Off]। *[ব্যাপারটা অনেকটা এরকম— দরজা বন্ধ করে দিয়ে বাইরে যাওয়ার অনুমতি দেয়া।]*



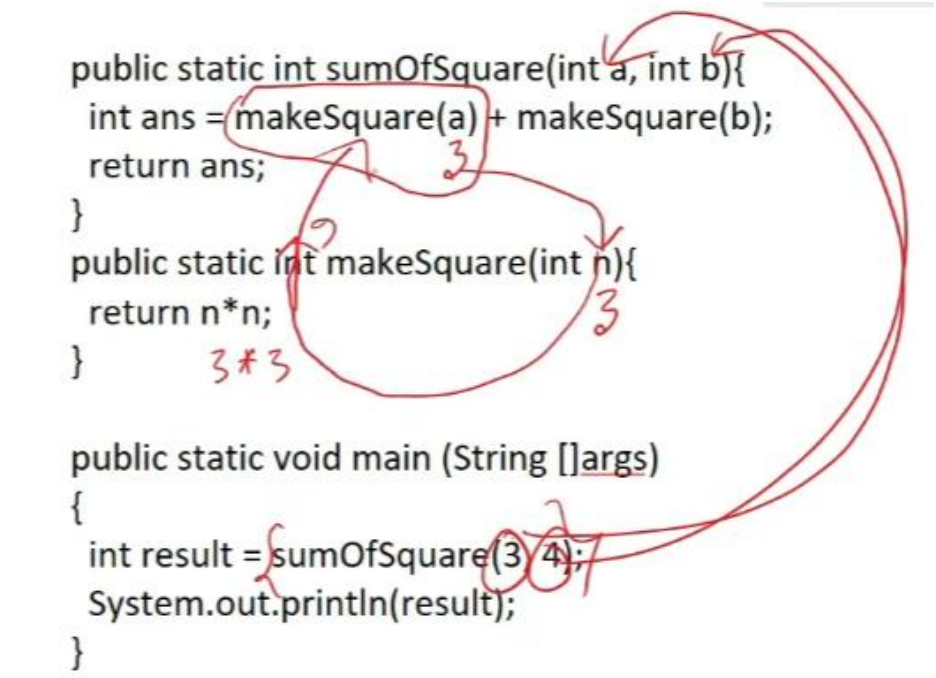
adder1 এর ক্ষেত্রে, a=4, b=5 receive করে result=4+5=9 print হয়ে চলে এসেছে। adder1 কিছু ফেরত দেয়নি। শুধু দেখিয়েছে।

কিন্তু adder2 এর ক্ষেত্রে, a=4, b=5 receive করে 4+5=9 হয়ে result variable এ জমা হলো। return result হওয়াতে সরাসরি output আসছে না। এই 9 টা line-8 এর int দিয়ে বের হবে এবং ফেরত আসবে adder2 (4, 5) জায়গায় অর্থাৎ, line-18 এ int ans = 9 হয়ে যাবে এবং পরবর্তী লাইনে ans অর্থাৎ 9 টা print করছি।

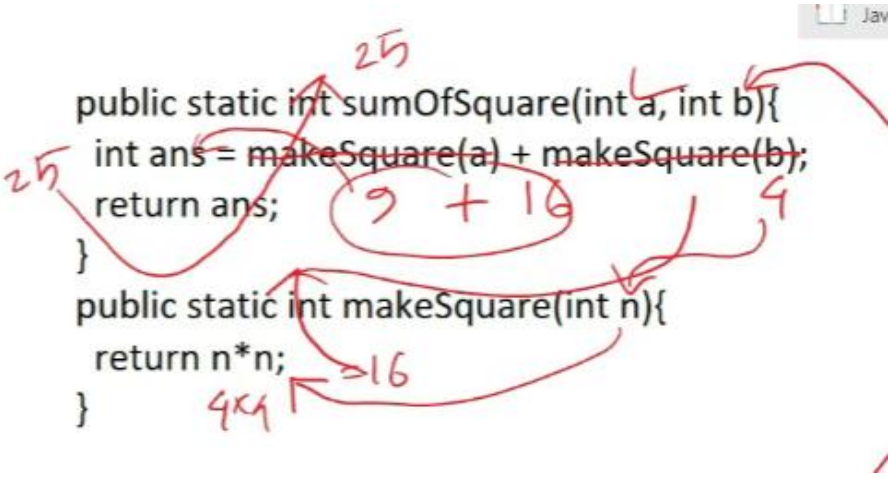
```java
import java.util.Scanner;
public class m4 {
    public static int sumOfSquare (int a, int b)
    {
        int ans = makeSquare(a) + makeSquare(b);
        return ans;
    }
    public static int makeSquare(int n)
    {
        return n*n;
    }
    Run | Debug
    public static void main (String [] args)
    {
        int result = sumOfSquare(a:3, b:4);
        System.out.println(result);
    }
}
```

```
PS F:\VS
doptium\j
' 'C:\Use
14b5\redh
25
PS F:\VS
```

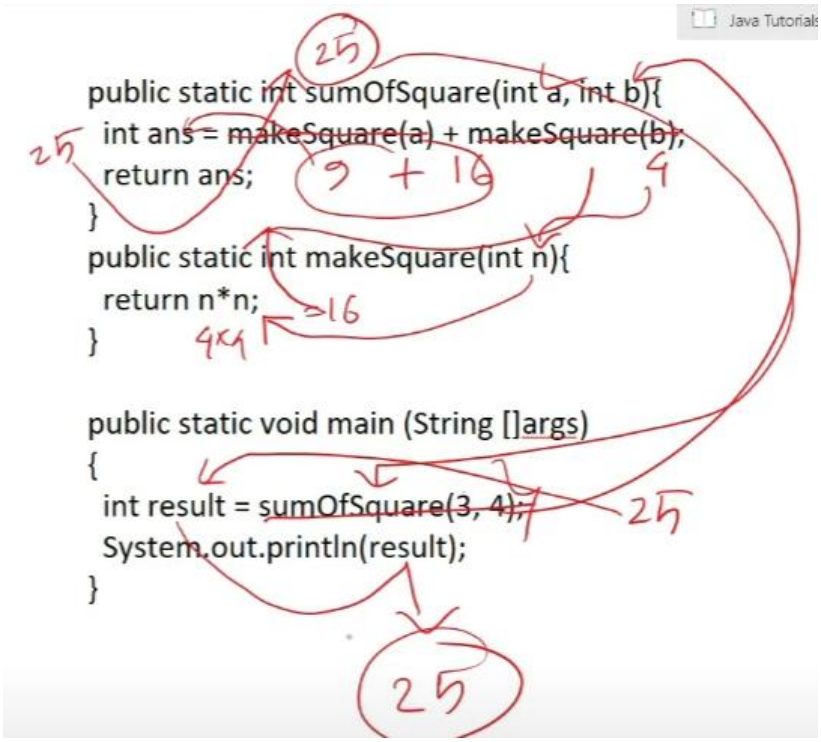
Step-1:

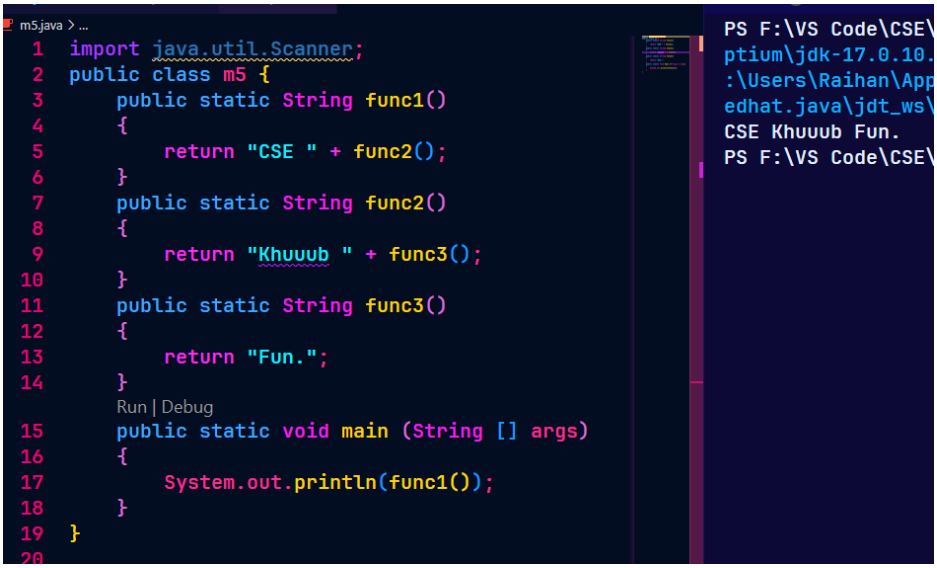


Step-2:

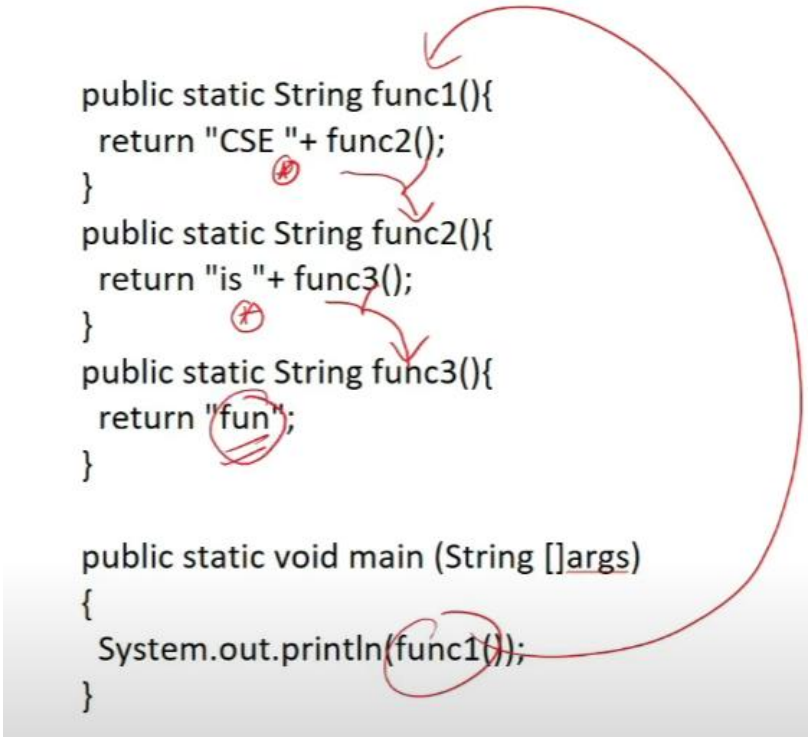


Step-3:

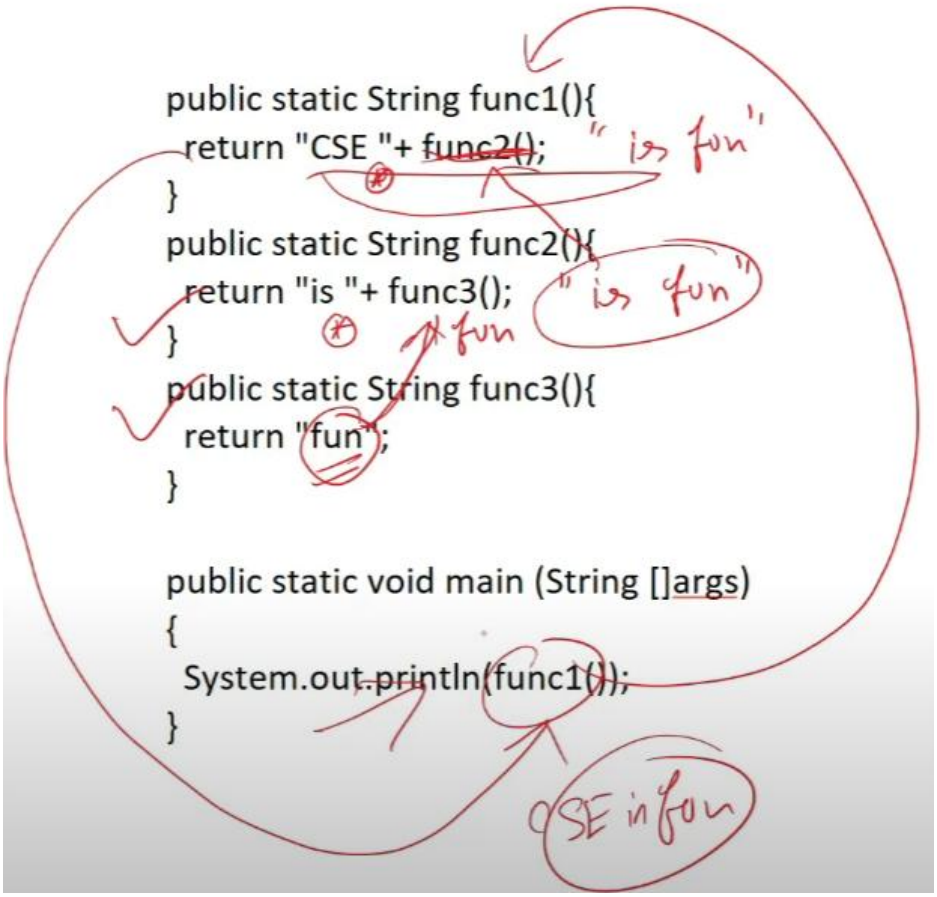

Java Tutorials
public static int sumOfSquare(int a, int b){
int ans = makeSquare(a) + makeSquare(b);
return ans;
}
public static int makeSquare(int n){
return n*n;
}
public static void main (String []args)
{
int result = sumOfSquare(3, 4);
System.out.println(result);
}
25
25
25
25
9 + 16
=16
4x4


m5.java > ...
1 import java.util.Scanner;
2 public class m5 {
3 public static String func1()
4 {
5 return "CSE " + func2();
6 }
7 public static String func2()
8 {
9 return "Khuuub " + func3();
10 }
11 public static String func3()
12 {
13 return "Fun.";
14 }
Run | Debug
15 public static void main (String [] args)
16 {
17 System.out.println(func1());
18 }
19 }
20
PS F:\VS Code\CSE\
ptium\jdk-17.0.10.
:\Users\Raihan\App
edhat.java\jdt_ws\
CSE Khuuub Fun.
PS F:\VS Code\CSE\

Step-1:


public static String func1(){
return "CSE "+ func2();
}
public static String func2(){
return "is "+ func3();
}
public static String func3(){
return "fun";
}
public static void main (String []args)
{
System.out.println(func1());
}

Step-2:


public static String func1(){
return "CSE "+ func2();
}
public static String func2(){
return "is "+ func3();
}
public static String func3(){
return "fun";
}
public static void main (String []args)
{
System.out.println(func1());
}
"is fun"
"is fun"
fun
CSE is fun

* * * *

Recursion

What is Recursion?

- Act of a method calling itself.
- Known as a recursive method.
- Will keep calling itself if there is no condition to stop.
- A way of problem solving by breaking down a broader problem into small parts.
- Mostly used as an alternative of loop especially when using loop can be quite difficult.
- Follows bottom to top then top to bottom approach (or can be considered vise versa as per convenience).
- Pros: You may have to write less code.
- Cons: Might have to deal with quite a few errors!

Loop versus Recursion

Loop	Recursion
Not much difference in terms of use. Both can be used to solve repetitive problems.	
Constructs like for, while used.	Method calls itself and consists of a base case and recursive case.
Loop condition given to stop loop execution at a point.	Stops executing when base case is reached.
May not be easy to write for some complex problems.	May be easier to write complex problems and have more clarity.
Is more memory efficient.	Possibility of using more memory and resources due to stack overflow error.

Components of Recursive Method

Two components:

- Base Case
- Termination condition of the recursion process.
- Recursive Case
- Other cases other than the base case where the method keeps calling itself.

Structure of a Recursive Method

```
static return_type recursive_method(datatype parameter){
    if (base case condition is satisfied){
        return final_value;
    }
    else{                           //recursive case
        recursive_method(argument);
    }
}
```

Method Call from Another Method

We all have seen in the previous “Method” lecture where we can call a method from another method.

Ex: Finding the sum of a simple series 1 + 2 + 3

- method1 will return the sum of the first two numbers to method2.
- method2 will add the returned value with 3 and return it to the method2 call in the main method where the returned result is printed.

```
public class MethodExample {
   public static void main(String[] args) {
       System.out.println(method2(3));
   }
   static int method1(int a, int b){
       int sum1 = a + b;
       return sum1;
   }
   static int method2(int b){
       int sum2 = method1(1, 2) + b;
       return sum2;
   }
}
```

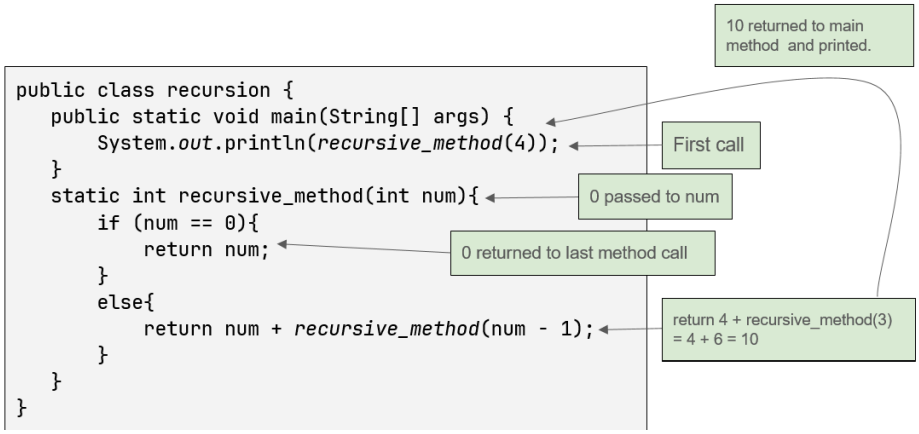
```
Welcome to DrJava. Working directory is
C:\Users\Desktop
6
```

Method Calling with Recursion

A recursive method works in a similar way. Except that the method will keep calling itself to keep adding the numbers and give the desired result.
And also, make sure the code works for sum of n natural numbers.
Ex: 1 + 2 + 3 + 4 + ... n
Change the number 4 in the method call to any other number and see if it works. You can also take n as user input.

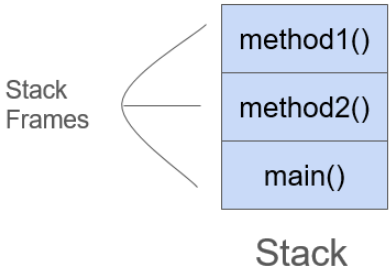
```
public class recursion {
   public static void main(String[] args) {
       System.out.println(recursive_method(4));
   }
   static int recursive_method(int num){
       if (num == 0){                //base case
           return num;
       }
       else{
            //recursive case
           return num + recursive_method(num - 1);
       }
   }
```

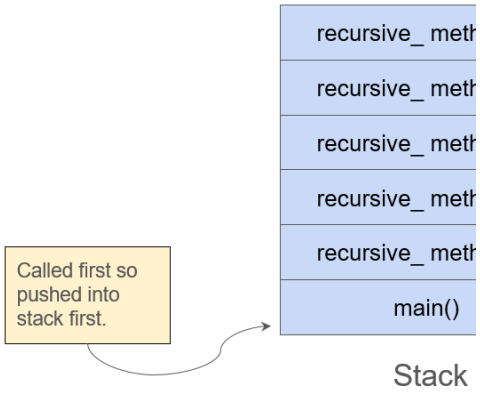
```
Welcome to DrJava. Working directory is
C:\Users\Desktop
10
```



Method Call Stack

- Used to keep track of method calls in a data structure called Stack.
- Each method, when called, gets allocated into the stack frame.
- Each stack frame consists of the information about the called method like its local variables, arguments passed, returned value etc.
- For recursive method, every method call is also allocated a stack frame.



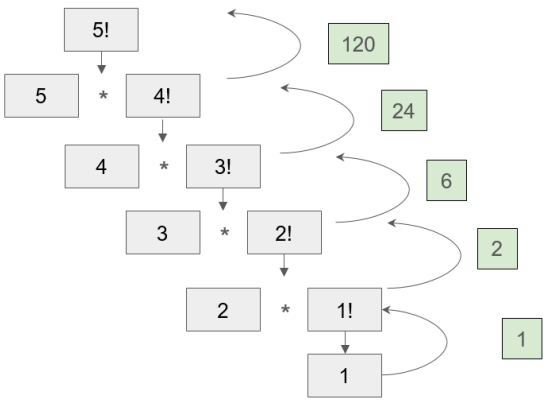


Factorial using Loop

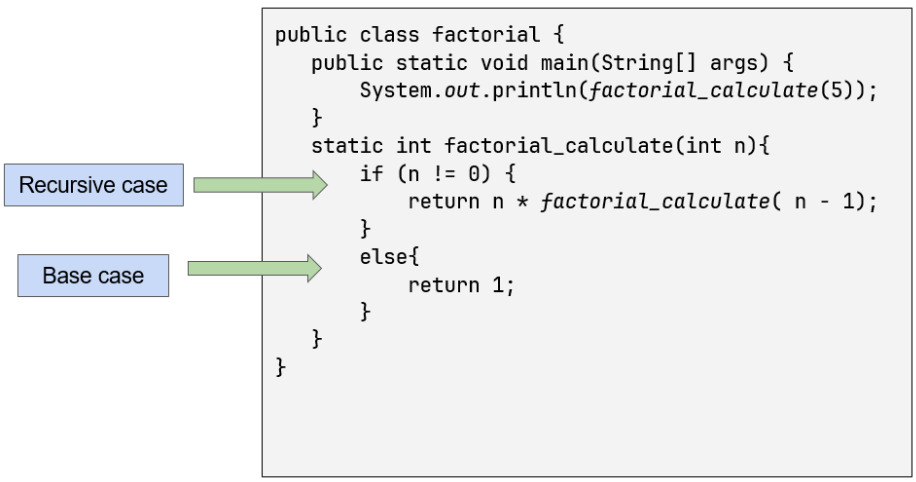
```
public class factorial {
   public static void main(String[] args)
{
        int fact = 1;
        int number = 5;
        for (int i = 1; i<=5; i++){
            fact *= i;
        }
        System.out.println(fact);
   }
}
```

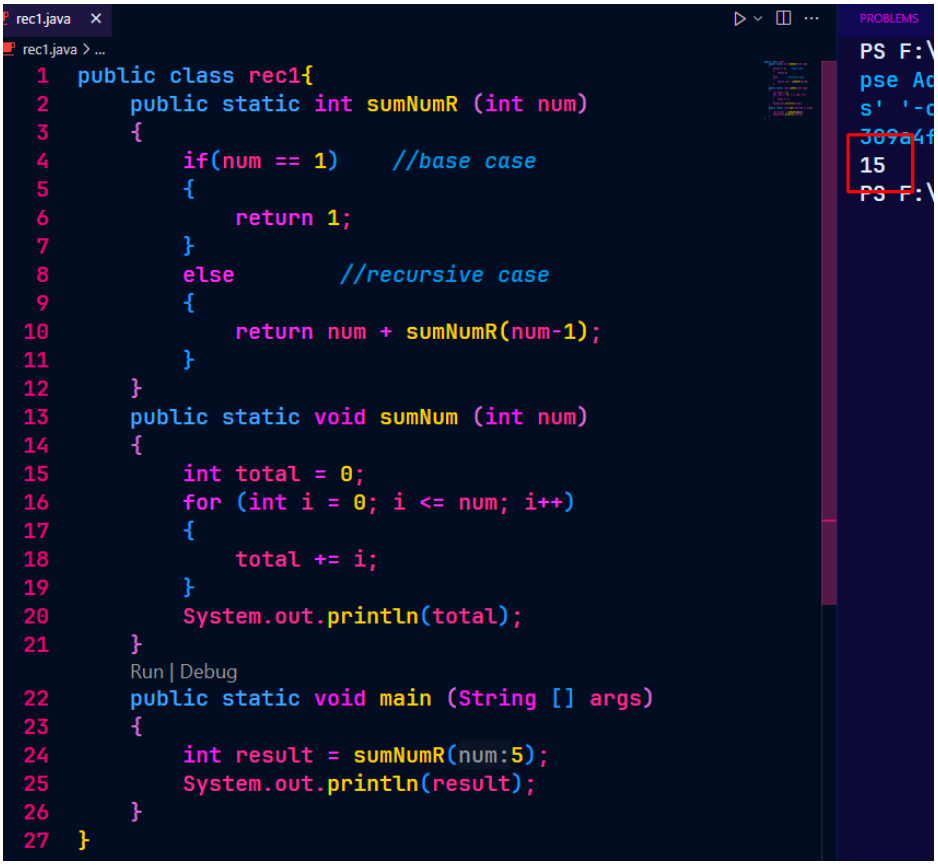
Factorial Problem Breakdown

Factorial of a number can be broken down into smaller parts.

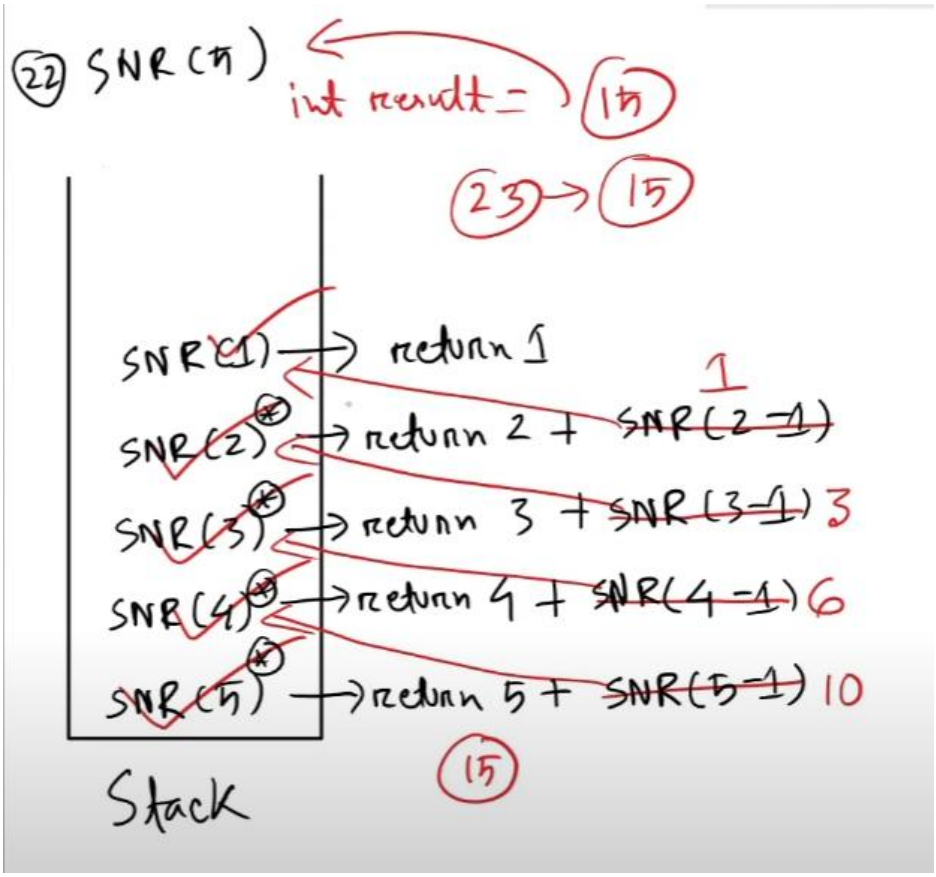


Factorial using Recursion




rec1.java
rec1.java > ...
1 public class rec1{
2 public static int sumNumR (int num)
3 {
4 if(num == 1) //base case
5 {
6 return 1;
7 }
8 else //recursive case
9 {
10 return num + sumNumR(num-1);
11 }
12 }
13 public static void sumNum (int num)
14 {
15 int total = 0;
16 for (int i = 0; i <= num; i++)
17 {
18 total += i;
19 }
20 System.out.println(total);
21 }
Run | Debug
22 public static void main (String [] args)
23 {
24 int result = sumNumR(num:5);
25 System.out.println(result);
26 }
27 }
PROBLEMS
PS F:\
pse Ac
s' '-c
15
PS F:\

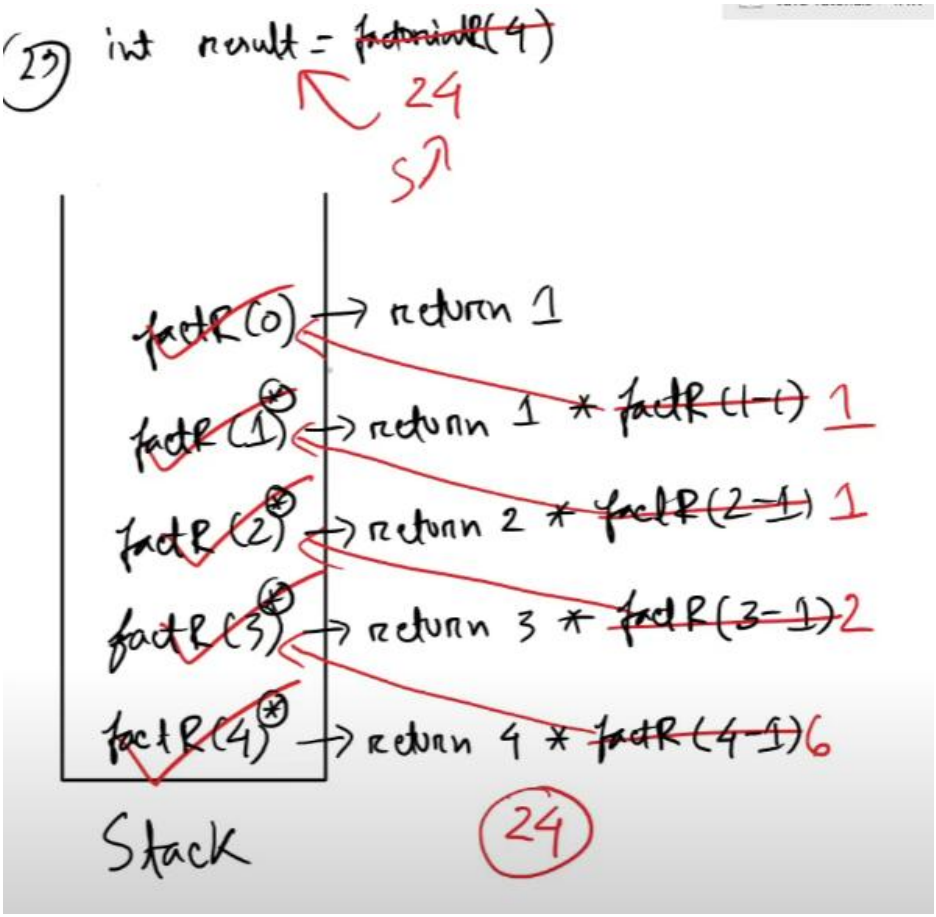
Explanation:




```java
public class rec2 {
    public static int factorialR (int num)
    {
        if (num == 0)   //base case
        {
            return 1;
        }
        else        //recursive case
        {
            return num * factorialR(num - 1);
        }
    }
    public static int factorial (int num)
    {
        int result = 1;
        if (num != 0)
        {
            for(int i = 1; i <= num; i++)
            {
                result = result * i;
            }
        }
        return result;
    }
    Run | Debug
    public static void main (String [] args)
    {
        int result = factorial(num:4);
        System.out.println(result);
    }
}
```

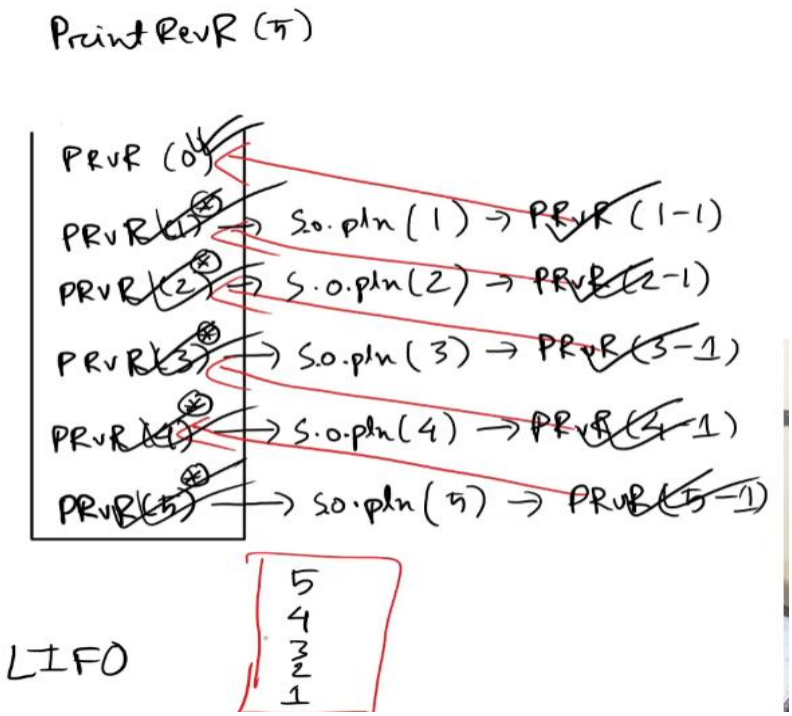
```
PS F:\VS
e Adopt
-cp' 'C
85a1c3c'
24
PS F:\VS
```

Explanation:



```java
public class rec3 {
    public static void printRevR(int num)
    {
        if (num == 0)    // base case
        {
            return;    // Ekhane kichui return hocche na.
                   // Method ta theke ber hoye jacche.
        }
        else
        {                   // recursive case
            System.out.println(num);
            printRevR(num-1);
        }
    }
    Run | Debug
    public static void main (String [] args)
    {
        printRevR(num:5);
    }
}
```

```
PS F:\
s\Ecli
ptionM
c00ab2
5
4
3
2
1
PS F:\
```



File Input/Output

Introduction

- File I/O refers to **File Input and Output** which is reading data from disk and writing data to files on disk/storage.
- This is a very important aspect of programming for tasks like:
  - Processing user data stored in hard disk.
  - Storing Application Data.
  - Loading a previous state of the application by reading from storage.
  - Reading Configuration Files.

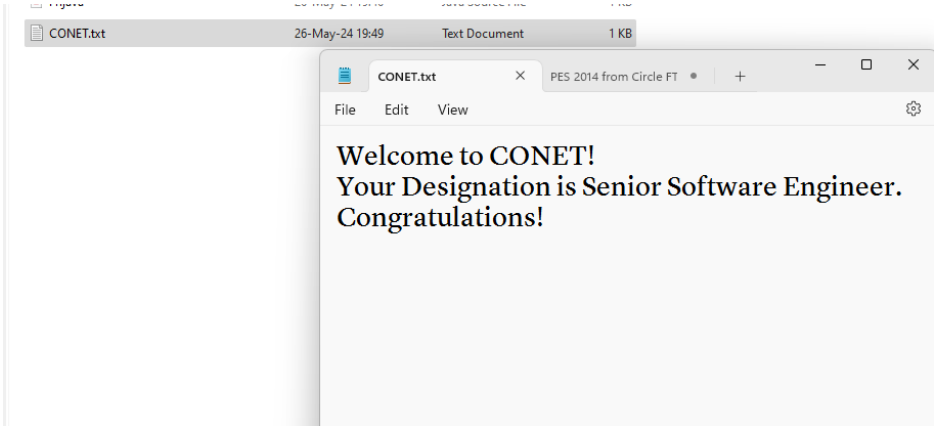
Reading/Writing files in java

There are few different ways to read/write files in Java. We'll be focusing on only one approach in this slide.

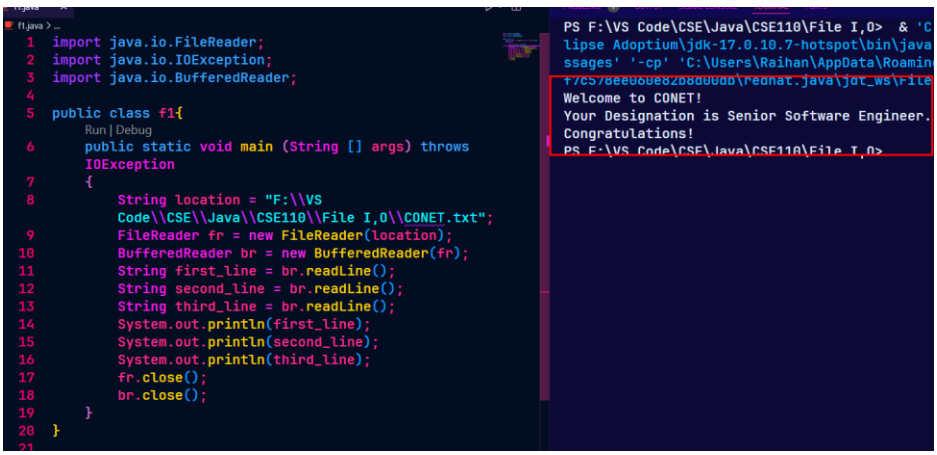
To read/write from a file we need to:

- Create a **FileReader/FileWriter object** to read/write the file.
- Then wrap it with a **BufferedReader/BufferedWriter object** for efficient reading/writing.

Reading files in java



এই text file কে read/পড়ার জন্যঃ

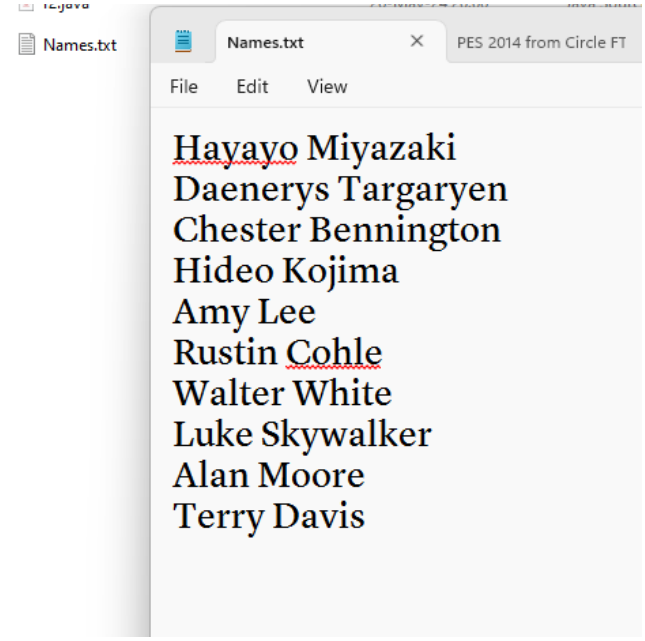


```
import java.io.FileReader;
import java.io.IOException;
import java.io.BufferedReader;

public class f1{
    Run | Debug
    public static void main (String [] args) throws
    IOException
    {
        String location = "F:\\VS
        Code\\CSE\\Java\\CSE110\\File I,0\\CONET.txt";
        FileReader fr = new FileReader(location);
        BufferedReader br = new BufferedReader(fr);
        String first_line = br.readLine();
        String second_line = br.readLine();
        String third_line = br.readLine();
        System.out.println(first_line);
        System.out.println(second_line);
        System.out.println(third_line);
        fr.close();
        br.close();
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE110\File I,0>  & 'C
lipse Adoptium\jdk-17.0.10.7-hotspot\bin\java
ssages' '-cp' 'C:\Users\Raihan\AppData\Roamin
Welcome to CONET!
Your Designation is Senior Software Engineer.
Congratulations!
PS F:\VS Code\CSE\Java\CSE110\File I,0>
```

Using loops to read lines from a file




```java
import java.io.FileReader;
import java.io.IOException;
import java.io.BufferedReader;

public class f2{
    public static void main (String [] args) throws
    IOException
    {
        String location = "F:\\VS
        Code\\CSE\\Java\\CSE110\\File I,O\\Names.txt";
        FileReader fr = new FileReader(location);
        BufferedReader br = new BufferedReader(fr);
        String line = br.readLine();
        int i = 1;

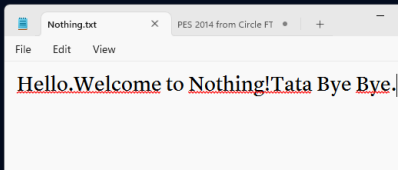
        while (line != null)
        {
            System.out.println("Name-" + i + " --> " +
            line);
            i++;
            line = br.readLine();
        }
        fr.close();
        br.close();
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE110\
clipse Adoptium\jdk-17.0.10.7-
Messages' '-cp' 'C:\Users\Raih
32af7c578ee060e82b8d00db\redha
Name-1 --> Hayayo Miyazaki
Name-2 --> Daenerys Targaryen
Name-3 --> Chester Bennington
Name-4 --> Hideo Kojima
Name-5 --> Amy Lee
Name-6 --> Rustin Cohle
Name-7 --> Walter White
Name-8 --> Luke Skywalker
Name-9 --> Alan Moore
Name-10 --> Terry Davis
PS F:\VS Code\CSE\Java\CSE110\
```

Writing files in java

```java
import java.io.FileWriter;
import java.io.IOException;
import java.io.BufferedWriter;

public class f3{
    public static void main (String [] args) throws IOException
    {
        String location = "F:\\VS Code\\CSE\\Java\\CSE110\\File I,O\\Nothing.txt";
        FileWriter fr = new FileWriter(location);
        BufferedWriter br = new BufferedWriter(fr);
        br.write(str:"Hello.");
        br.write(str:"Welcome to Nothing!");
        br.write(str:"Tata Bye Bye.");
        br.close();
        fr.close();
    }
}
```

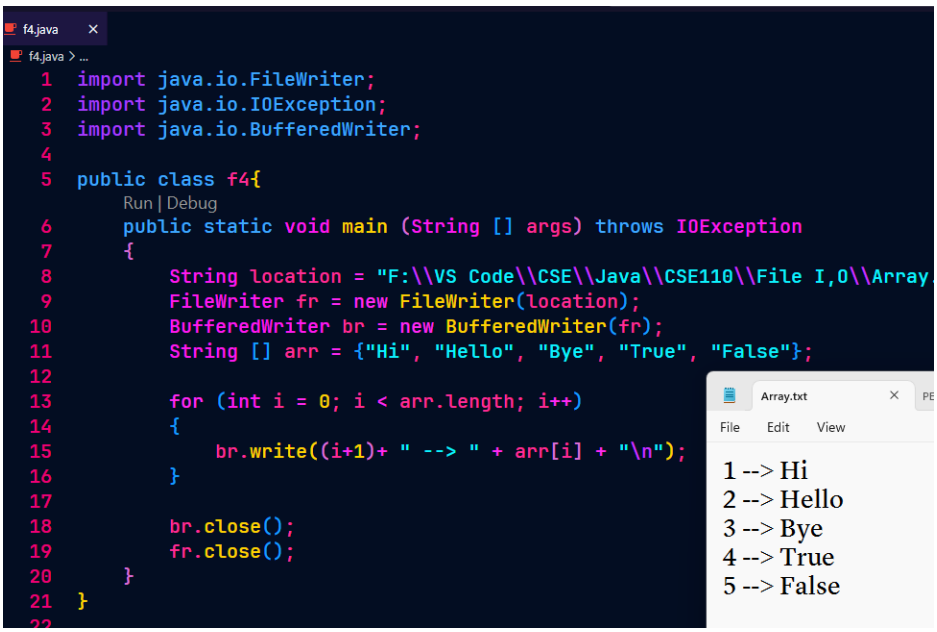


Read করার সময় আলাদা আলাদা লাইনে ছিলো কিন্তু write করলে একই লাইনে চলে আসছে।

সেজন্য প্রতি br.write( ) এ লেখার সাথে \n দিয়ে দিবো।



Using loops to write lines to a file



There are two main reasons why you need to close FileReader and BufferedReader in Java:

(i) Releasing System Resources to ensure efficient resource management on your system :

Both FileReader and BufferedReader creates connections to underlying system resources.
When you open a file for reading, the operating system allocates resources like to manage the connection.

If you don't close these using close( ), these resources remain occupied even after your program finishes reading the file.
This can lead to resource leaks. Closing them ensures releasing these resources back to operating system for other uses.

(ii) Ensuring Data Consistency and integrity:

Closing the BufferedReader ensures everything in the buffer gets written to the file. This guarantees all the information you've read is properly saved in the file.

Without closing, there's a chance that some data might remain in the buffers and not be written to the file, leading to inconsistencies between what your program thinks it wrote and the actual content on the file.
মূলকথা হলো, যখন FileReader/Writer & BufferedReader/Writer এর প্রক্রিয়া শুরু হয় তখন তারা কিছু Resources

Allocate (বরাদ্দ) করে রাখে। তাই Resource গুলোকে free করার জন্য br.close( ),fr.close( ) ব্যবহার করা হয়।

উদাহরণ হিসেবে যদি বলি, একটা পানির কল ছেড়ে দেয়ার পর যদি সেটা বন্ধ না করা হয় তাহলে তো পানি পড়তেই থাকবে, যা একটি সমস্যা। তাই পানি ব্যবহারের পর অবশ্যই কল বন্ধ করতে হবে।